

Lecture Notes in Adaptive Filters

Second Edition

Jesper Kjær Nielsen
jkn@es.aau.dk
Aalborg University

Søren Holdt Jensen
shj@es.aau.dk
Aalborg University

Last revised: October 16, 2014

Nielsen, Jesper Kjær and Jensen, Søren Holdt
Lecture Notes in Adaptive Filters

Copyright © 2011-2012 Jesper Kjær Nielsen and Søren Holdt Jensen, except where otherwise stated.
All rights reserved.

1. edition: June 7, 2011
2. edition: September 19, 2012

Department of Electronic Systems
Aalborg University
Fredrik Bajers Vej 7
DK-9220 Aalborg Ø
Denmark

These lecture notes have been printed with Computer Modern 10pt and been typeset using L^AT_EX 2_ε on a computer running the GNU/Linux operating system. All of the figures have been created using GNUPLOT, PGF and the macro packages TikZ and PGFPLOTS. Simulations have been run in MatlabTM.

Contents

Preface	v
1 Introduction, Wiener-Hopf Equations, and Normal Equations	1
1.1 Review of the Basics	1
1.1.1 Linear Algebra	1
1.1.2 Optimisation	2
1.1.3 Stochastic Processes	3
1.2 Block Diagram of Adaptive Filtering	5
1.2.1 Mean-Squared Error and Squared Error cost functions	5
1.3 The Wiener-Hopf Equations	6
1.3.1 Principle of Orthogonality	7
1.3.2 The Modified Yule-Walker Equations	7
1.4 The Normal Equations	7
1.4.1 Principle of Orthogonality	8
1.4.2 Estimation of the (Cross-)Correlation	9
1.4.3 Data Windowing	9
2 Steepest Descent and Least-Mean-Square Adaptive Filters	11
2.1 Review of the Basics	11
2.1.1 The Eigenvalue Decomposition	11
2.2 The Wiener-Hopf Equations	12
2.3 The Method of Steepest Descent	15
2.3.1 Basic Idea	15
2.3.2 Transient Analysis	15
2.4 Least-Mean-Square Adaptive Filters	17
2.4.1 Basic Idea	17
2.4.2 Transient Analysis	18
2.4.3 Steady-State Analysis	20
2.4.4 Computational Cost	21
3 Normalised LMS and Affine Projection Algorithm	23
3.1 Review of the Basics	23
3.1.1 Inversion of a 2 by 2 Block Matrix	23
3.1.2 The Method of Lagrange Multipliers	23
3.2 Overview over Adaptive Filters based on the Mean-Squared Error Cost Function	25
3.2.1 Model for the Analysis of SGMs	26
3.2.2 How to Analyse Adaptive Filters	27
3.3 LMS Revisited	28

3.3.1	Transient Analysis	28
3.3.2	Steady-State Analysis	29
3.4	Normalised LMS Adaptive Filters	30
3.4.1	Transient Analysis	30
3.4.2	Steady-State Analysis	31
3.4.3	Computational Cost	31
3.4.4	Another Derivation of the NLMS Algorithm	31
3.5	Affine Projection Adaptive Filters	32
3.5.1	Transient Analysis	34
3.5.2	Steady-State Analysis	34
3.5.3	Computational Cost	34
4	Recursive Least-Squares Adaptive Filters	35
4.1	Review of the Basics	35
4.1.1	The Matrix Inversion Lemma	35
4.2	Method of Least-Squares	35
4.2.1	Weighted Least-Squares	36
4.2.2	Weight Functions	38
4.3	The Recursive Least-Squares Algorithm with an Exponential Weight Function	40
4.3.1	Selection of the Forgetting Factor	42
4.3.2	Transient Analysis	43
4.3.3	Steady-State Analysis	43
4.3.4	Computational Cost	44
	Bibliography	45
	A Summary	47
B	Transient and Steady-State Analysis of the LMS Adaptive Filter	51
B.1	A Special Fourth Order Moment of a Gaussian Random Vector	51
B.2	The Analysis Model	52
B.3	Transient Analysis	52
B.3.1	Mean-Square Convergence	52
B.3.2	Learning Curve	58
B.4	Steady-State Analysis	58
B.4.1	Mean-Square Deviation	58
B.4.2	Excess Mean-Square Error	60
B.4.3	Misadjustment	61

Preface

```
\*
* Your warranty is now void.
*
* By installing this knowledge onto your brain, you accept that we, the
* authors of the present lecture notes, are not responsible for potential
* confusion, wasted time, failed exams, or misguided missiles.
*
* You will most likely encounter unclear statements and remarks, poor English
* usage, and errors when you read the present notes. Therefore, use these
* notes at your own risk, and please help us correct these problems by giving
* us some feedback.
*\
```

The present lecture notes were written for the annual course on adaptive filters at Aalborg University. The content of the course is now a part of the annual course called *Array and Sensor Signal Processing*. The notes are written for the lecturer, but they may also be useful to the student as a supplement to his/her favourite textbook. Consequently, the notes are very concise and contain only what we believe to be the basics of adaptive filtering. Moreover, we have also made some important simplifications.

1. We use real-valued numbers and not complex-valued numbers. Although the latter is more general, it is less confusing and leads to fewer errors when real-valued numbers are used.
2. We only consider FIR adaptive filters.
3. The signals have zero mean. This is a standard assumption used in most textbooks.

Each of the lectures contains an amount of material suited for a lecture lasting for approximately 90 minutes. The appendices contain a summary and some supplementary material.

These lecture notes are always work in progress. Therefore, if you have found an error, have a suggestion for a better statement and/or explanation, or just want to give us some feedback, then do not hesitate to contact us.

Lecture 1

Introduction, Wiener-Hopf Equations, and Normal Equations

1.1 Review of the Basics

1.1.1 Linear Algebra

Notation

An N -dimensional vector is written as

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} = [x_1 \quad x_2 \quad \cdots \quad x_N]^T \quad (1.1)$$

where $(\cdot)^T$ denotes the transpose. An $N \times M$ -dimensional matrix is written as

$$\mathbf{A} = \begin{bmatrix} a_{11} & \cdots & a_{1M} \\ \vdots & \ddots & \vdots \\ a_{N1} & \cdots & a_{NM} \end{bmatrix}. \quad (1.2)$$

Matrix Properties

Let $\mathbf{A}$ be a square matrix. We then have that

$$\mathbf{A} \text{ is orthogonal} \iff \mathbf{A}^T = \mathbf{A}^{-1} \iff \mathbf{I} = \mathbf{A}\mathbf{A}^T = \mathbf{A}^T\mathbf{A} \quad (1.3)$$

$$\mathbf{A} \text{ is symmetric} \iff \mathbf{A}^T = \mathbf{A} \quad (1.4)$$

$$\mathbf{A} \text{ is skew-symmetric} \iff \mathbf{A}^T = -\mathbf{A}. \quad (1.5)$$

Let $Q(\mathbf{x}) = \mathbf{x}^T \mathbf{A} \mathbf{x}$ where $\mathbf{A}$ is a square and symmetric matrix. We then have that

$$\begin{aligned} \mathbf{A} \text{ is positive definite (p.d.)} &\iff Q(\mathbf{x}) > 0 \quad \forall \mathbf{x} \neq 0 \\ \mathbf{A} \text{ is positive semidefinite (p.s.d.)} &\iff Q(\mathbf{x}) \geq 0 \quad \forall \mathbf{x} \\ \mathbf{A} \text{ is negative definite (n.d.)} &\iff Q(\mathbf{x}) < 0 \quad \forall \mathbf{x} \neq 0 \\ \mathbf{A} \text{ is negative semidefinite (n.s.d.)} &\iff Q(\mathbf{x}) \leq 0 \quad \forall \mathbf{x} \\ \mathbf{A} \text{ is indefinite otherwise.} & \end{aligned}$$

The interested reader can find more on linear algebra in for example [1] and [2].

1.1.2 Optimisation

Gradient and Hessian

Let $f(\mathbf{x})$ be a scalar function with continuous second-order partial derivatives. The gradient vector of $f(\mathbf{x})$ is defined as

$$\mathbf{g}(\mathbf{x}) = \nabla f(\mathbf{x}) = \frac{\partial f}{\partial \mathbf{x}} = \left[\frac{\partial f}{\partial x_1} \quad \frac{\partial f}{\partial x_2} \quad \cdots \quad \frac{\partial f}{\partial x_N} \right]^T. \quad (1.6)$$

The Hessian matrix of $f(\mathbf{x})$ is symmetric and defined as

$$\begin{aligned} \mathbf{H}(\mathbf{x}) &= \nabla[\mathbf{g}^T(\mathbf{x})] = \nabla[\nabla^T f(\mathbf{x})] = \frac{\partial^2 f}{\partial \mathbf{x} \partial \mathbf{x}^T} \\ &= \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_N} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_N \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_N^2} \end{bmatrix}. \end{aligned} \quad (1.7)$$

When $f(\mathbf{x})$ consists of a constant term, we have that

$$f(\mathbf{x}) = c \implies \begin{cases} \mathbf{g}(\mathbf{x}) = \mathbf{0} \\ \mathbf{H}(\mathbf{x}) = \mathbf{0} \end{cases}.$$

When $f(\mathbf{x})$ consists of a linear term, we have that

$$f(\mathbf{x}) = \mathbf{v}^T \mathbf{x} = \mathbf{x}^T \mathbf{v} \implies \begin{cases} \mathbf{g}(\mathbf{x}) = \mathbf{v} \\ \mathbf{H}(\mathbf{x}) = \mathbf{0} \end{cases}.$$

When $f(\mathbf{x})$ consists of a quadratic term, we have that

$$f(\mathbf{x}) = \mathbf{x}^T \mathbf{A} \mathbf{x} \implies \begin{cases} \mathbf{g}(\mathbf{x}) = (\mathbf{A} + \mathbf{A}^T) \mathbf{x} = 2\mathbf{A} \mathbf{x} \\ \mathbf{H}(\mathbf{x}) = \mathbf{A} + \mathbf{A}^T = 2\mathbf{A} \end{cases}$$

where the last equality for the gradient and the Hessian holds if and only if $\mathbf{A}$ is symmetric.

Unconstrained Optimisation

We can solve an unconstrained optimisation problem by following a five step recipe.

1. Construct the cost function $f(\mathbf{x})$.

2. Find the gradient $\mathbf{g}(\mathbf{x}) = \frac{\partial f}{\partial \mathbf{x}}$.
3. Solve $\mathbf{g}(\mathbf{x}) = \mathbf{0}$ for $\mathbf{x}$. The solutions $\{\mathbf{x}_i\}$ are called critical points.
4. Find the Hessian $\mathbf{H}(\mathbf{x}) = \frac{\partial^2 f}{\partial \mathbf{x} \partial \mathbf{x}^T}$ and compute it for all the critical points. If

$$\begin{aligned}
\mathbf{H}(\mathbf{x}_i) \text{ is p.d.} &\implies f(\mathbf{x}_i) \text{ is a local minimum} \\
\mathbf{H}(\mathbf{x}_i) \text{ is n.d.} &\implies f(\mathbf{x}_i) \text{ is a local maximum} \\
\mathbf{H}(\mathbf{x}_i) \text{ is indefinite} &\iff f(\mathbf{x}_i) \text{ is a saddle point} \\
\mathbf{H}(\mathbf{x}_i) \text{ is p.s.d or n.s.d} &\iff \text{further analysis is necessary [3, p. 41]}
\end{aligned}$$

5. Find the global minimiser/maximiser by evaluating $f(\mathbf{x}_i)$ for every critical point.

If $f(\mathbf{x})$ is a convex¹ function, we have the following useful fact [3, p. 57]

$$f(\mathbf{x}) \text{ is a convex function} \iff \mathbf{H}(\mathbf{x}) \text{ is p.d. } \forall \mathbf{x} .$$

Thus, there is only one critical point corresponding to a minimum if and only if $f(\mathbf{x})$ is a convex function.

The interested reader can find more on optimisation in for example [3] and [4].

1.1.3 Stochastic Processes

Let $X(n)$ for $n = n_0, n_0 + 1, \dots, n_0 + N - 1$ be a stochastic process with joint probability density function (pdf) $p_X(x(n_0), x(n_0 + 1), \dots, x(n_0 + N - 1))$.

Mean, Covariance and Correlation

The mean, covariance and correlations sequences are defined as

$$\text{Mean sequence:} \quad \mu_X(n) = E[X(n)] = \int x(n) p_X(x(n)) dx(n) \quad (1.8)$$

$$\begin{aligned}
\text{Covariance sequence:} \quad c_X(n, n+k) &= \text{cov}(X(n), X(n+k)) \\
&= E[(X(n) - \mu_X(n))(X(n+k) - \mu_X(n+k))] \quad (1.9)
\end{aligned}$$

$$\text{Correlation sequence:} \quad r_X(n, n+k) = E[X(n)X(n+k)] , \quad (1.10)$$

and they are related by

$$c_X(n, n+k) = r_X(n, n+k) - \mu_X(n)\mu_X(n+k) . \quad (1.11)$$

For a finite number of observation, we may define the mean vector

$$\boldsymbol{\mu}_X = [\mu_X(n_0) \quad \mu_X(n_0 + 1) \quad \cdots \quad \mu_X(n_0 + N - 1)]^T , \quad (1.12)$$

the covariance matrix

$$\mathbf{C}_X = \begin{bmatrix} c_X(n_0, n_0) & \cdots & c_X(n_0, n_0 + N - 1) \\ \vdots & \ddots & \vdots \\ c_X(n_0 + N - 1, n_0) & \cdots & c_X(n_0 + N - 1, n_0 + N - 1) \end{bmatrix} , \quad (1.13)$$

¹If we should be completely accurate, we should distinguish between weakly and strictly convex functions. However, in this note, we only use convexity in the strict sense.

and the correlation matrix

$$\mathbf{R}_X = \begin{bmatrix} r_X(n_0, n_0) & \cdots & r_X(n_0, n_0 + N - 1) \\ \vdots & \ddots & \vdots \\ r_X(n_0 + N - 1, n_0) & \cdots & r_X(n_0 + N - 1, n_0 + N - 1) \end{bmatrix}. \quad (1.14)$$

They are related by

$$\mathbf{C}_X = \mathbf{R}_X - \boldsymbol{\mu}_X \boldsymbol{\mu}_X^T. \quad (1.15)$$

$\mathbf{C}_X$ and $\mathbf{R}_X$ are symmetric and p.s.d. They are p.d. if $X(n)$ is *not* perfectly predictable.

Stationarity and Wide Sense Stationarity (WSS)

$$\begin{aligned} X(n) \text{ is stationary} &\iff p_X(x(n_0 + m), x(n_0 + 1 + m), \dots, x(n_0 + N - 1 + m)) \\ &\quad \downarrow \\ &\quad = p_X(x(n_0), x(n_0 + 1), \dots, x(n_0 + N - 1)) \quad \forall n_0, m, N \\ X(n) \text{ is WSS} &\iff \begin{cases} \mu_X(n) = \mu_X & (\text{a constant}) \\ r_X(n, n + k) = r_X(0, k) & (\text{a function of } k \text{ not } n) \end{cases} \end{aligned}$$

Since $r_X(0, k)$ does not depend on n , it is often written as $r_X(k)$. A Venn diagram for all stochastic processes is shown in Fig. 1.1.

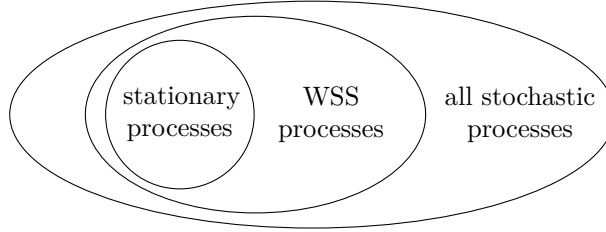


Figure 1.1: A Venn diagram for all stochastic processes.

Let $Y(n)$ be another stochastic process.

$$X(n) \text{ and } Y(n) \text{ are jointly WSS} \iff \begin{cases} X(n) \text{ is WSS} \\ Y(n) \text{ is WSS} \\ r_{X,Y}(n, n + k) = r_{X,Y}(0, k) \end{cases}$$

where $r_{X,Y}(n, n + k)$ is the cross-correlation sequence.

Estimation of Statistics

Often, we wish to estimate the statistics of $X(n)$ from a single realisation $x(n)$. If $X(n)$ is (wide sense) stationary and *ergodic*² in some sense (for example, in mean, in correlation, in power, or in distribution), we may estimate some or all of its statistics from a single realisation $x(n)$. Examples are the unbiased estimators of the mean

$$\hat{\mu}_X = \frac{1}{N} \sum_{n=n_0}^{n_0+N-1} x(n) \quad (1.16)$$

²Ergodicity basically means that we can infer something about the statistics of a stochastic process from a single realisation of it.

and the correlation sequence

$$\hat{r}_X(k) = \frac{1}{N-k} \sum_{n=n_0}^{n_0+N-1-k} x(n)x(n+k) . \quad (1.17)$$

The interested reader can find more on stochastic processes in for example [5] and [6].

1.2 Block Diagram of Adaptive Filtering

The adaptive filtering problem is shown in Fig. 1.2. From the figure, we have that

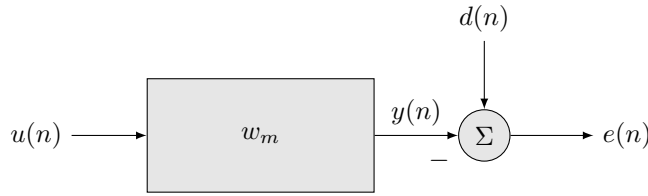


Figure 1.2: Block Diagram of Adaptive Filtering in a WSS environment.

$u(n)$: zero-mean, WSS input signal

w_m : M -tap FIR-filter with impulse response $w_0, w_1, \dots, w_{M-1}$

$y(n)$: output signal given by

$$y(n) = \sum_{m=0}^{M-1} w_m u(n-m) \quad (1.18)$$

$d(n)$: zero-mean, WSS desired signal

$e(n)$: error signal

Moreover, $u(n)$ and $d(n)$ are assumed to be jointly WSS.

1.2.1 Mean-Squared Error and Squared Error cost functions

Define

$$\mathbf{w} = [w_0 \ w_1 \ \dots \ w_{M-1}]^T \quad (1.19)$$

$$\mathbf{u}(n) = [u(n) \ u(n-1) \ \dots \ u(n-M+1)]^T \quad (1.20)$$

$$\mathbf{R}_u = E[\mathbf{u}(n)\mathbf{u}^T(n)] \quad (1.21)$$

$$\mathbf{r}_{ud} = E[\mathbf{u}(n)d(n)] . \quad (1.22)$$

Here, $\mathbf{R}_u$ is the correlation matrix of the input signal vector $\mathbf{u}(n)$, and $\mathbf{r}_{ud}$ is the cross-correlation vector between the input signal vector $\mathbf{u}(n)$ and the desired signal $d(n)$.

For $n = n_0, n_0 + 1, \dots, n_0 + K - 1$ with $K \geq M$, define

$$\mathbf{A} = [\mathbf{u}(n_0) \quad \mathbf{u}(n_0 + 1) \quad \cdots \quad \mathbf{u}(n_0 + K - 1)]^T \quad (1.23)$$

$$\mathbf{d} = [d(n_0) \quad d(n_0 + 1) \quad \cdots \quad d(n_0 + K - 1)]^T \quad (1.24)$$

$$\mathbf{e} = [e(n_0) \quad e(n_0 + 1) \quad \cdots \quad e(n_0 + K - 1)]^T. \quad (1.25)$$

Then

$$e(n) = d(n) - \mathbf{u}^T(n)\mathbf{w} \iff d(n) = \mathbf{u}^T(n)\mathbf{w} + e(n) \quad (1.26)$$

and

$$\mathbf{e} = \mathbf{d} - \mathbf{A}\mathbf{w} \iff \mathbf{d} = \mathbf{A}\mathbf{w} + \mathbf{e}. \quad (1.27)$$

We observe the input signal $u(n)$ and the desired signal $d(n)$, and we wish to find the filter vector $\mathbf{w}$ which minimises the error in some sense. Two popular choices for the cost function of this problem are

$$\text{Mean-squared error:} \quad J_1(\mathbf{w}) = E[e(n)^2] \quad (\text{statistics is known}) \quad (1.28)$$

$$\text{Squared error:} \quad J_2(\mathbf{w}) = \sum_{n=n_0}^{n_0+K-1} e^2(n) = \mathbf{e}^T \mathbf{e} \quad (\text{statistics is unknown}) \quad (1.29)$$

If $\text{rank}(\mathbf{A}) > M$, Eq. (1.27) constitutes an overdetermined system of equations which do not have an exact solution that makes $J_2(\mathbf{w}) = 0$. The problem of minimising the squared error is often referred to as *the method of least-squares*, and the argument minimising the squared error is referred to as *the least-squares solution*. Moreover, the squared error divided by the number of elements in the error vector $\mathbf{e}$ can be seen as an unbiased estimate of the mean-squared error when $u(n)$ and $d(n)$ are assumed to be jointly WSS. That is,

$$E \left[\frac{1}{K} J_2(\mathbf{w}) \right] = J_1(\mathbf{w}). \quad (1.30)$$

1.3 The Wiener-Hopf Equations

We want to solve the following unconstrained optimisation problem

$$\mathbf{w}_o = \arg \min_{\mathbf{w}} J_1(\mathbf{w}) \quad (1.31)$$

where $\mathbf{w}_o$ is the vector containing the optimal filter coefficients. We use the five step recipe in Sec. 1.1.2 to solve the optimisation problem.

1. Construct the cost function

$$\begin{aligned} J_1(\mathbf{w}) &= E[e(n)^2] = E[(d(n) - \mathbf{u}^T(n)\mathbf{w})^2] = E[(d(n) - \mathbf{u}^T(n)\mathbf{w})^T (d(n) - \mathbf{u}^T(n)\mathbf{w})] \\ &= E[d(n)^2] + E[\mathbf{w}^T \mathbf{u}(n) \mathbf{u}^T(n) \mathbf{w}] - E[d(n) \mathbf{u}^T(n) \mathbf{w}] - E[\mathbf{w}^T \mathbf{u}(n) d(n)] \\ &= E[d(n)^2] + \mathbf{w}^T E[\mathbf{u}(n) \mathbf{u}^T(n)] \mathbf{w} - 2\mathbf{w}^T E[\mathbf{u}(n) d(n)] \\ &= \sigma_d^2 + \mathbf{w}^T \mathbf{R}_u \mathbf{w} - 2\mathbf{w}^T \mathbf{r}_{ud} \quad (\text{quadratic cost function}) \end{aligned} \quad (1.32)$$

2. Find the gradient

$$\mathbf{g}(\mathbf{w}) = (\mathbf{R}_u + \mathbf{R}_u^T) \mathbf{w} - 2\mathbf{r}_{ud} = 2\mathbf{R}_u \mathbf{w} - 2\mathbf{r}_{ud} \quad (1.33)$$

3. Solve $\mathbf{g}(\mathbf{w}) = \mathbf{0}$ for $\mathbf{w}$

$$\begin{aligned} \mathbf{g}(\mathbf{w}) &= 2\mathbf{R}_u\mathbf{w} - 2\mathbf{r}_{ud} = \mathbf{0} \\ \Downarrow \\ \boxed{\mathbf{R}_u\mathbf{w} = \mathbf{r}_{ud}} & \quad (\text{Wiener-Hopf Equations}) \end{aligned} \quad (1.34)$$

$$\Downarrow \\ \mathbf{w} = \mathbf{R}_u^{-1}\mathbf{r}_{ud} \quad (\text{If } \mathbf{R}_u \text{ is invertible}) \quad (1.35)$$

4. Find the Hessian

$$\mathbf{H}(\mathbf{w}) = 2\mathbf{R}_u \quad (1.36)$$

which is p.d. for all $\mathbf{w}$ if $u(n)$ is not perfectly predictable.

5. This implies that

- $J_1(\mathbf{w})$ is a convex function,
- $\mathbf{R}_u$ is invertible,
- $\mathbf{w}_o = \mathbf{R}_u^{-1}\mathbf{r}_{ud}$ is the global minimiser, and
- $J_1(\mathbf{w}_o) = \sigma_d^2 - \mathbf{r}_{ud}^T \mathbf{R}_u^{-1} \mathbf{r}_{ud}$.

The solution $\mathbf{w}_o$ is often referred to as the least-mean-squares solution.

1.3.1 Principle of Orthogonality

The Wiener-Hopf equations can also be derived from the principle of orthogonality. For $m = 0, 1, \dots, M-1$, the principle of orthogonality states

$$0 = E[u(n-m)e_o(n)] \iff 0 = E[y_o(n)e_o(n)] \iff \mathbf{w}_o = \mathbf{R}_u^{-1}\mathbf{r}_{ud}. \quad (1.37)$$

where $e_o(n)$ and $y_o(n)$ are the error and the filter output, respectively, when the filter is optimised in the mean-squared sense.

1.3.2 The Modified Yule-Walker Equations

The Wiener-Hopf equations are closely related to the Modified Yule-Walker equations. To see this, let $d(n) = u(n+1)$. Then

$$\mathbf{r}_{ud} = E[\mathbf{u}(n)d(n)] = E[\mathbf{u}(n)u(n+1)] = \mathbf{p}_u \quad (1.38)$$

which implies that

$$\mathbf{R}_u\mathbf{w} = \mathbf{p}_u \quad (\text{Modified Yule-Walker Equations}). \quad (1.39)$$

1.4 The Normal Equations

We want to solve the following unconstrained optimisation problem

$$\mathbf{w}_o = \arg \min_{\mathbf{w}} J_2(\mathbf{w}) \quad (1.40)$$

where $\mathbf{w}_o$ is the vector containing the optimal filter coefficients. We use the five step recipe in Sec. 1.1.2 to solve the optimisation problem.

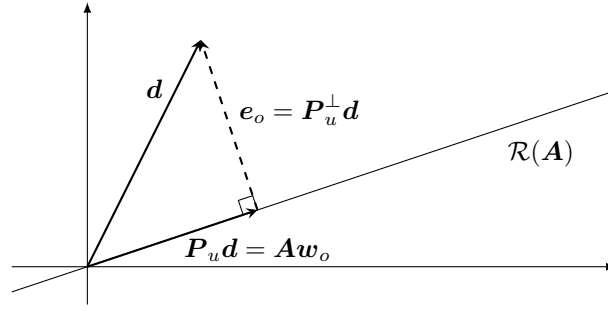


Figure 1.3: The principle of orthogonality for the squared error cost function.

1. Construct the cost function

$$\begin{aligned} J_2(\mathbf{w}) &= \mathbf{e}^T \mathbf{e} = (\mathbf{d} - \mathbf{A}\mathbf{w})^T (\mathbf{d} - \mathbf{A}\mathbf{w}) \\ &= \mathbf{d}^T \mathbf{d} + \mathbf{w}^T \mathbf{A}^T \mathbf{A} \mathbf{w} - 2\mathbf{w}^T \mathbf{A}^T \mathbf{d} \quad (\text{quadratic cost function}) \end{aligned} \quad (1.41)$$

2. Find the gradient

$$\mathbf{g}(\mathbf{w}) = (\mathbf{A}^T \mathbf{A} + \mathbf{A}^T \mathbf{A})\mathbf{w} - 2\mathbf{A}^T \mathbf{d} = 2\mathbf{A}^T \mathbf{A} \mathbf{w} - 2\mathbf{A}^T \mathbf{d} \quad (1.42)$$

3. Solve $\mathbf{g}(\mathbf{w}) = \mathbf{0}$ for $\mathbf{w}$

$$\begin{aligned} \mathbf{g}(\mathbf{w}) &= 2\mathbf{A}^T \mathbf{A} \mathbf{w} - 2\mathbf{A}^T \mathbf{d} = \mathbf{0} \\ \Downarrow \\ \boxed{\mathbf{A}^T \mathbf{A} \mathbf{w} = \mathbf{A}^T \mathbf{d}} & \quad (\text{Normal Equations}) \end{aligned} \quad (1.43)$$

$$\Downarrow \quad \mathbf{w} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{d} \quad (\text{If } \mathbf{A} \text{ has full rank}) \quad (1.44)$$

4. Find the Hessian

$$\mathbf{H}(\mathbf{w}) = 2\mathbf{A}^T \mathbf{A} \quad (1.45)$$

which is p.d. for all $\mathbf{w}$ if $\mathbf{A}$ has full rank.

5. This implies that

- $J_2(\mathbf{w})$ is a convex function,
- $\mathbf{w}_o = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{d}$ is the global minimiser, and
- $J_2(\mathbf{w}_o) = \mathbf{d}^T (\mathbf{I} - \mathbf{P}_u) \mathbf{d} = \mathbf{d}^T \mathbf{P}_u^\perp \mathbf{d}$ where $\mathbf{P}_u = \mathbf{A}(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$ is the orthogonal projection matrix and $\mathbf{P}_u^\perp$ is the complementary projection matrix to $\mathbf{P}_u$.

The solution $\mathbf{w}_o$ is often referred to as the least-squares solution.

1.4.1 Principle of Orthogonality

The normal equations can also be derived from the principle of orthogonality

$$\mathbf{0} = \mathbf{A}^T \mathbf{e}_o \iff \mathbf{0} = \mathbf{y}_o^T \mathbf{e}_o \iff \mathbf{w}_o = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{d} . \quad (1.46)$$

where $\mathbf{e}_o$ and $\mathbf{y}_o$ are the error vector and the filter output vector, respectively, when the filter is optimised in the squared sense. Fig. 1.3 illustrates the principle of orthogonality.

1.4.2 Estimation of the (Cross-)Correlation

Comparing the normal equations with the Wiener-Hopf equations, we see that

$$\hat{\mathbf{R}}_u = \frac{1}{K} \mathbf{A}^T \mathbf{A} \quad (1.47)$$

$$\hat{\mathbf{r}}_{ud} = \frac{1}{K} \mathbf{A}^T \mathbf{d} \quad (1.48)$$

are the build in estimates of the correlation matrix and the cross-correlation vector, respectively, in the method of least-squares.

1.4.3 Data Windowing

Recall the definition of $\mathbf{A}^T$ in Eq. (1.23) with $n_0 = 1$ and $K = N + M - 1$.

$$\left[\begin{array}{ccc|ccc|ccc} u(1) & \cdots & u(M-1) & u(M) & \cdots & u(N) & u(N+1) & \cdots & u(N+M-1) \\ u(0) & \cdots & u(M-2) & u(M-1) & \cdots & u(N-1) & u(N) & \cdots & u(N+M-2) \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ u(2-M) & \cdots & u(0) & u(1) & \cdots & u(N-M+1) & u(N-M+2) & \cdots & u(N) \end{array} \right]$$

In real-world applications, we only observe a finite amount of data. Assume that we observe $u(n)$ for $n = 1, \dots, N$ while the remaining data for $n \leq 0$ and $n > N$ are unobserved. If we set the unobserved data equal to zero, we obtain the structure for $\mathbf{A}$ shown in Fig 1.4. The various

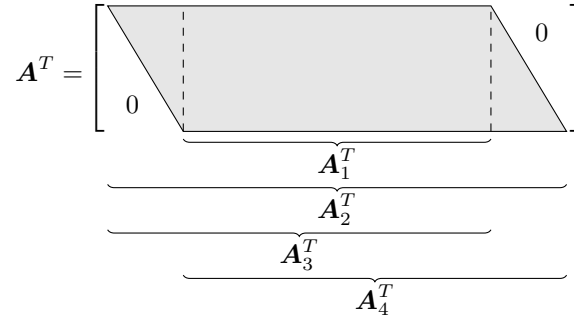


Figure 1.4: Four different data windowing methods.

choices for $\mathbf{A}$ are

$\mathbf{A}_1$ - covariance method:	$n_0 = M$	$K = N - M + 1$
$\mathbf{A}_2$ - autocorrelation method:	$n_0 = 1$	$K = N + M - 1$
$\mathbf{A}_3$ - prewindowing method:	$n_0 = 1$	$K = N$
$\mathbf{A}_4$ - postwindowing method:	$n_0 = M$	$K = N$

The cost function and the length of $\mathbf{e}$ in Eq. (1.25) and $\mathbf{d}$ in Eq. (1.24) must be adjusted according to the windowing method. The covariance method leads to unbiased estimates of $\mathbf{R}_u$ and $\mathbf{r}_{ud}$ in Eq. (1.47) and Eq. (1.48), respectively.

Lecture 2

Steepest Descent and Least-Mean-Square Adaptive Filters

2.1 Review of the Basics

2.1.1 The Eigenvalue Decomposition

If a square $M \times M$ matrix $\mathbf{A}$ has an eigenvalue decomposition (EVD), it can be written as

$$\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1} \quad \Longleftrightarrow \quad \mathbf{A}\mathbf{X} = \mathbf{X}\mathbf{\Lambda} \quad (2.1)$$

where $\mathbf{X}$ contains the M linearly independent eigenvectors of $\mathbf{A}$, and the diagonal matrix $\mathbf{\Lambda}$ contains the M eigenvalues $\{\lambda_m\}_{m=1}^M$ of $\mathbf{A}$. If $\mathbf{A}$ does not have an eigenvalue decomposition, it is said to be *defective*.

Matrix Powers

Let $\mathbf{A}$ have an EVD. Then

$$\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1} \quad (2.2)$$

$$\mathbf{A}^2 = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}\mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1} = \mathbf{X}\mathbf{\Lambda}^2\mathbf{X}^{-1} \quad (2.3)$$

$$\vdots$$
$$\mathbf{A}^n = \mathbf{X}\mathbf{\Lambda}^n\mathbf{X}^{-1} . \quad (2.4)$$

EVD of Special Matrices

Let $\mathbf{A}$ be a symmetric matrix. Then

$\mathbf{A}$ has an EVD,

$\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^T$ ($\mathbf{X}$ is an orthogonal matrix), and

$\lambda_m \in \mathbb{R}$ for $m = 1, \dots, M$

Let $\mathbf{A}$ be a p.d. matrix. Then

$$\lambda_m \in \mathbb{R}^+ \text{ for } m = 1, \dots, M.$$

where $\mathbb{R}^+$ is the set of all positive real numbers.

Matrix Trace

The trace of $\mathbf{A}$ is

$$\text{tr}(\mathbf{A}) = \sum_{m=1}^M a_{mm} = \sum_{m=1}^M \lambda_m = \text{tr}(\mathbf{\Lambda}) . \quad (2.5)$$

Condition Number of a Normal Matrix

If $\mathbf{A}$ is a normal matrix (i.e., $\mathbf{A}^T \mathbf{A} = \mathbf{A} \mathbf{A}^T$), then the condition number of $\mathbf{A}$ is

$$\kappa(\mathbf{A}) = \left| \frac{\lambda_{\max}}{\lambda_{\min}} \right| = \chi(\mathbf{A}) \quad (2.6)$$

where $\chi(\mathbf{A})$ is the eigenvalue spread of $\mathbf{A}$.

2.2 The Wiener-Hopf Equations

The adaptive filtering problem with time-varying filter coefficients is shown in Fig. 2.1. From

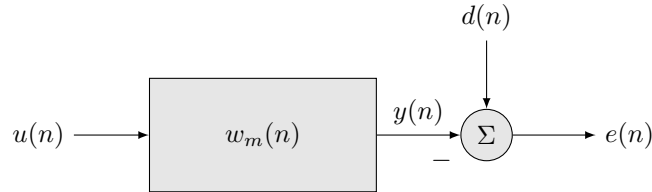


Figure 2.1: Block diagram of adaptive filtering in a non-WSS environment.

the figure, we have that

$u(n)$: zero-mean input signal

$w_m(n)$: M -tap FIR-filter with impulse response $w_0(n), w_1(n), \dots, w_{M-1}(n)$

$y(n)$: output signal given by $y_n = \sum_{m=0}^{M-1} w_m(n)u(n-m)$

$d(n)$: zero-mean desired signal

$e(n)$: error signal

WSS Signals

When $u(n)$ and $d(n)$ are jointly WSS, the filter coefficients are not time-varying. Define

$$\mathbf{w} = [w_0 \ w_1 \ \cdots \ w_{M-1}]^T \quad (2.7)$$

$$\mathbf{u}(n) = [u(n) \ u(n-1) \ \cdots \ u(n-M+1)]^T. \quad (2.8)$$

Then

$$e(n) = d(n) - y(n) = d(n) - \mathbf{u}^T(n)\mathbf{w}. \quad (2.9)$$

We wish to minimise

$$\begin{aligned} J_1(\mathbf{w}) &= E[e(n)^2] = E[d(n)^2] + \mathbf{w}^T E[\mathbf{u}(n)\mathbf{u}^T(n)]\mathbf{w} - 2\mathbf{w}^T E[\mathbf{u}(n)d(n)] \\ &= \sigma_d^2 + \mathbf{w}^T \mathbf{R}_u \mathbf{w} - 2\mathbf{w}^T \mathbf{r}_{ud} \end{aligned} \quad (2.10)$$

w.r.t. $\mathbf{w}$. The minimiser

$$\mathbf{w}_o = \mathbf{R}_u^{-1} \mathbf{r}_{ud} \quad (2.11)$$

is the unique solution to the Wiener-Hopf equations

$$\mathbf{R}_u \mathbf{w} = \mathbf{r}_{ud}, \quad (2.12)$$

provided that $\mathbf{R}_u$ is p.d.

Non-WSS Signals

If $u(n)$ and $d(n)$ are *not* jointly WSS, the optimal filter coefficients are time-varying, and we have that

$$\mathbf{R}_u(n)\mathbf{w}(n) = \mathbf{r}_{ud}(n) \quad (2.13)$$

where

$$\mathbf{R}_u(n) = \begin{bmatrix} r_u(n, n) & \cdots & r_u(n, n-M+1) \\ \vdots & \ddots & \vdots \\ r_u(n-M+1, n) & \cdots & r_u(n-M+1, n-M+1) \end{bmatrix} \quad (2.14)$$

$$\mathbf{r}_{ud}(n) = [r_{ud}(n, n) \ \cdots \ r_{ud}(n-M+1, n)]^T. \quad (2.15)$$

We could find the optimal solution by calculating

$$\mathbf{w}_o(n) = \mathbf{R}_u^{-1}(n)\mathbf{r}_{ud}(n) \quad (2.16)$$

for every time index n . However, this approach may suffer from problems such as that

1. the computational complexity is high, and
2. the statistics is unknown.

The steepest descent algorithm solves the first problem, and the least-mean-square (LMS) adaptive filter solves both problems.

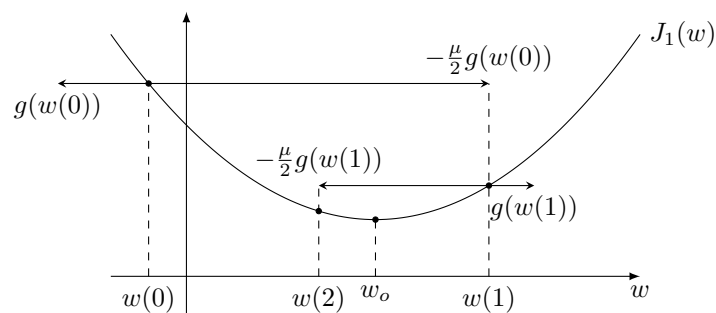


Figure 2.2: The first three iterations of the steepest descent algorithm for a one-dimensional filter vector.

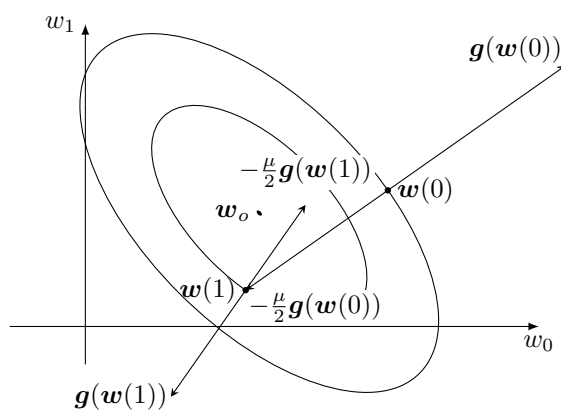


Figure 2.3: The first three iterations of the steepest descent algorithm for a two-dimensional filter vector.

2.3 The Method of Steepest Descent

2.3.1 Basic Idea

The basic idea of the steepest descent (SD) algorithm is to find the unique solution $\mathbf{w}_o$ of the Wiener-Hopf equations through a series of steps, starting from some point $\mathbf{w}(0)$. The steps are taken in the opposite direction of the gradient $\mathbf{g}(\mathbf{w}(n))$. This idea is illustrated in Fig. 2.2 and Fig. 2.3. We update the filter coefficients by a recursive update equation given by

$$\boxed{\mathbf{w}(n+1) = \mathbf{w}(n) - \frac{\mu}{2} \mathbf{g}(\mathbf{w}(n))} \quad (2.17)$$

where μ is the step-size and $\mathbf{g}(\mathbf{w}(n))$ is the gradient of the cost function $J_1(\mathbf{w}(n))$. The gradient is

$$\mathbf{g}(\mathbf{w}(n)) = 2\mathbf{R}_u(n)\mathbf{w}(n) - 2\mathbf{r}_{ud}(n) . \quad (2.18)$$

Since the gradient is a deterministic function, the evolution of the filter coefficient vector is also a deterministic function.

Define the weight error

$$\Delta\mathbf{w}(n) = \mathbf{w}_o - \mathbf{w}(n) . \quad (2.19)$$

In order for the SD algorithm to converge to the solution $\mathbf{w}_o$, we must require that the step-size μ is selected such that

$$\lim_{n \rightarrow \infty} \Delta\mathbf{w}(n) = 0 \quad (2.20)$$

when $u(n)$ and $d(n)$ are jointly WSS. If this is true, the SD algorithm is said to be *stable*. Moreover, we would like to select μ such that the $\Delta\mathbf{w}(n)$ becomes small as fast as possible.

2.3.2 Transient Analysis

Assume that $u(n)$ and $d(n)$ are jointly WSS. We then have that

$$\Delta\mathbf{w}(n) = \mathbf{w}_o - \left(\mathbf{w}(n-1) - \frac{\mu}{2} \mathbf{g}(\mathbf{w}(n-1)) \right) \quad (2.21)$$

$$= \Delta\mathbf{w}(n-1) + \mu(\mathbf{R}_u\mathbf{w}(n-1) - \mathbf{R}_u\mathbf{w}_o) \quad (2.22)$$

$$= (\mathbf{I} - \mu\mathbf{R}_u)\Delta\mathbf{w}(n-1) \quad (2.23)$$

$$= (\mathbf{I} - \mu\mathbf{R}_u)(\mathbf{I} - \mu\mathbf{R}_u)\Delta\mathbf{w}(n-2) \quad (2.24)$$

$$= (\mathbf{I} - \mu\mathbf{R}_u)^n \Delta\mathbf{w}(0) . \quad (2.25)$$

Since the correlation matrix $\mathbf{R}_u$ is symmetric, it has an eigenvalue decomposition $\mathbf{R}_u = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^T$ where $\mathbf{X}$ is an orthogonal matrix. That is, we may write

$$\Delta\mathbf{w}(n) = \mathbf{X}(\mathbf{I} - \mu\mathbf{\Lambda})^n \mathbf{X}^T \Delta\mathbf{w}(0) \quad (2.26)$$

where $\mathbf{I} - \mu\mathbf{\Lambda}$ is a diagonal matrix with the m 'th diagonal given by the mode $1 - \mu\lambda_m$. For Eq. (2.20) to be fulfilled from any starting point $\mathbf{w}(0)$, we must therefore require that

$$\lim_{n \rightarrow \infty} (1 - \mu\lambda_m)^n = 0 , \quad \text{for } m = 1, 2, \dots, M . \quad (2.27)$$

Thus

$$-1 < 1 - \mu\lambda_m < 1 , \quad \text{for } m = 1, 2, \dots, M \quad (2.28)$$

$\Updownarrow$

$$-1 < 1 - \mu\lambda_{\max} < 1 \quad (2.29)$$

Solving for μ leads to that the SD algorithm is stable if

$$0 < \mu < \frac{2}{\lambda_{\max}} . \quad (2.30)$$

From Eq. (2.20), we see that

$$\begin{aligned} |1 - \mu\lambda_m| \text{ is close to } 0 & \iff \text{mode with fast convergence} \\ |1 - \mu\lambda_m| \text{ is close to } 1 & \iff \text{mode with slow convergence} \end{aligned}$$

The optimal step-size μ_o is therefore given by

$$\mu_o = \arg \min_{\mu} \max_{\lambda_m} |1 - \mu\lambda_m| \quad (2.31)$$

$$\text{subject to } |1 - \mu\lambda_m| < 1, \text{ for } m = 1, \dots, M . \quad (2.32)$$

That is, μ_o minimises the value of the largest (slowest) mode. We solve this optimisation problem by the use of Fig 2.4. From Fig 2.4, we see that the optimal step-size μ_o satisfies

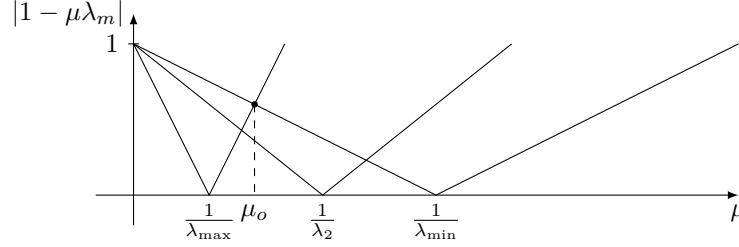


Figure 2.4: Finding the optimal step-size.

$$\begin{aligned} 1 - \mu_o \lambda_{\min} &= -(1 - \mu_o \lambda_{\max}) \\ \iff \end{aligned} \quad (2.33)$$

$$\boxed{\mu_o = \frac{2}{\lambda_{\max} + \lambda_{\min}}} . \quad (2.34)$$

For the optimal step-size, the slowest modes $\pm(1 - \mu_o \lambda_{\max})$ and $\pm(1 - \mu_o \lambda_{\min})$ are therefore given by

$$\pm \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \pm \frac{\kappa(\mathbf{R}_u) - 1}{\kappa(\mathbf{R}_u) + 1} \quad (2.35)$$

where $\kappa(\mathbf{R}_u)$ is the condition number of the correlation matrix $\mathbf{R}_u$. Thus, if the condition number is large, the slowest modes are close to one, and the convergence of the SD algorithm is slow. Conversely, if the condition number is close to one, the slowest modes are close to zero, and the convergence of the SD algorithm is fast.

Learning Curve

The cost function can be written as

$$J_1(\mathbf{w}(n)) = J_1(\mathbf{w}_o) + \Delta \mathbf{w}^T(n) \mathbf{R}_u \Delta \mathbf{w}(n) \quad (2.36)$$

$$= J_1(\mathbf{w}_o) + \Delta \mathbf{w}^T(0) \mathbf{X} (\mathbf{I} - \mu \mathbf{\Lambda})^n \mathbf{\Lambda} (\mathbf{I} - \mu \mathbf{\Lambda})^n \mathbf{X}^T \Delta \mathbf{w}(0) \quad (2.37)$$

$$= J_1(\mathbf{w}_o) + \sum_{m=1}^M \lambda_m (1 - \mu \lambda_m)^{2n} (\mathbf{x}_m^T \Delta \mathbf{w}(0))^2 . \quad (2.38)$$

The plot of $J_1(\mathbf{w}(n))$ as a function of n is called the learning curve, and a sketch of it is shown in Fig. 2.5.

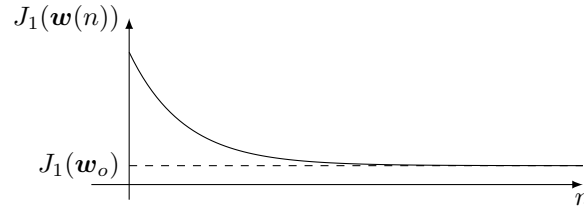


Figure 2.5: The learning curve of the steepest descent algorithm.

2.4 Least-Mean-Square Adaptive Filters

2.4.1 Basic Idea

Recall, that the gradient of the mean-squared error cost function $J_1(\mathbf{w}(n))$ is

$$\mathbf{g}(\mathbf{w}(n)) = \frac{\partial E[e^2(n)]}{\partial \mathbf{w}(n)} = 2E \left[\frac{\partial e(n)}{\partial \mathbf{w}(n)} e(n) \right] = -2E[\mathbf{u}(n)e(n)] . \quad (2.39)$$

Often, we do not know the gradient, and we therefore have to estimate it. A simple estimate is

$$\hat{\mathbf{g}}(\mathbf{w}(n)) = -2\mathbf{u}(n)e(n) . \quad (2.40)$$

If we replace the gradient in the steepest descent algorithm with this estimate, we obtain

$$\boxed{\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \mathbf{u}(n)e(n)} \quad (2.41)$$

which is called the least-mean-square (LMS) algorithm. A block diagram of the LMS filter is depicted in Fig. 2.6. The LMS algorithm is the simplest *stochastic gradient method* (SGM). The

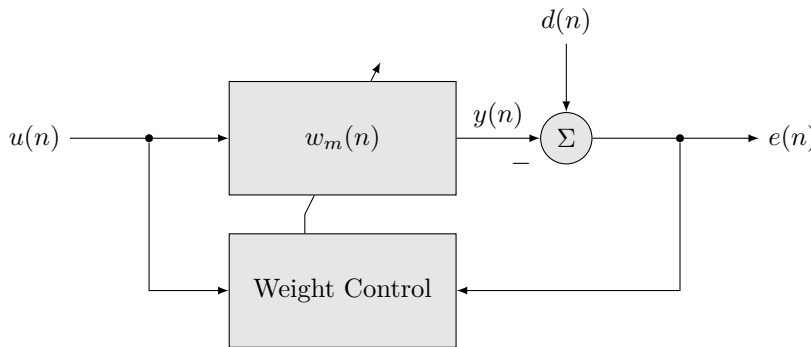


Figure 2.6: Typical block diagram of an adaptive filter in the case where the statistics is unknown.

naming of the SGM refers to that the gradient is a stochastic process. This has the following consequences.

1. The filter coefficient vector $\mathbf{w}(n)$ is also a stochastic process. This makes the analysis of the SGMs difficult. To see this, consider the cost function

$$\begin{aligned} J_1(\mathbf{w}(n)) &= E[e^2(n)] \\ &= \sigma_d^2(n) + E[\mathbf{w}^T(n)\mathbf{u}(n)\mathbf{u}^T(n)\mathbf{w}(n)] - 2E[\mathbf{w}^T(n)\mathbf{u}(n)d(n)] . \end{aligned} \quad (2.42)$$

For the stochastic gradient methods, it is not easy to evaluate the expectations in the cost function since $\mathbf{w}(n)$ and $\mathbf{u}(n)$ are not independent (unless $M = 1$ and $u(n)$ is a white process).

2. The weight error $\Delta\mathbf{w}(n) = \mathbf{w}_o - \mathbf{w}(n)$ never goes permanently to zero. That is, in steady-state ($n \rightarrow \infty$), the filter coefficient vector $\mathbf{w}(\infty)$ fluctuates randomly around the optimum $\mathbf{w}_o$. Consequently, the mean-square error (MSE) $J_1(\mathbf{w}(\infty))$ in steady-state is larger than $J_{\min} = J_1(\mathbf{w}_o)$ by an amount referred to as the *excess mean-square* error (EMSE)

$$J_{\text{ex}} = J_1(\mathbf{w}(\infty)) - J_{\min} . \quad (2.43)$$

The ratio of the EMSE to the MSE is called the *misadjustment*

$$\mathcal{M} = \frac{J_{\text{ex}}}{J_{\min}} = \frac{J_1(\mathbf{w}(\infty))}{J_{\min}} - 1 . \quad (2.44)$$

In Lecture 3, we say more about these parameters and the analysis of adaptive filters.

2.4.2 Transient Analysis

Assume that $d(n)$ is given by¹

$$d(n) = \mathbf{u}^T(n)\mathbf{w}_o + v(n) \quad (2.45)$$

where $v(n)$ is white Gaussian noise with variance σ_v^2 and uncorrelated with $u(n)$. Moreover, assume that the random variables of the stochastic process $\mathbf{u}(n)$ are independent and identically distributed (IID) with a Gaussian distribution and correlation matrix $\mathbf{R}_u$. This assumption is infeasible unless $M = 1$. Nevertheless, we make this assumption anyway in order to make the transient analysis as simple as possible. We are concerned with

$$\begin{aligned} \lim_{n \rightarrow \infty} E[\Delta\mathbf{w}(n)] &= 0 && \text{(Convergence in mean)} \\ \lim_{n \rightarrow \infty} E[\|\Delta\mathbf{w}(n)\|^2] &= c < \infty && \text{(Convergence in mean-square)} \end{aligned}$$

where $\|\Delta\mathbf{w}(n)\|^2 = \Delta\mathbf{w}^T(n)\Delta\mathbf{w}(n)$ is the vector 2-norm, and c is some positive constant².

Convergence in the Mean

By subtracting the LMS recursion from the optimal filter coefficients, we obtain

$$\Delta\mathbf{w}(n) = \mathbf{w}_o - [\mathbf{w}(n-1) + \mu\mathbf{u}(n-1)e(n-1)] \quad (2.46)$$

$$= \Delta\mathbf{w}(n-1) - \mu\mathbf{u}(n-1) [d(n-1) - \mathbf{u}^T(n-1)\mathbf{w}(n-1)] \quad (2.47)$$

$$= [\mathbf{I} - \mu\mathbf{u}(n-1)\mathbf{u}^T(n-1)] \Delta\mathbf{w}(n-1) - \mu\mathbf{u}(n-1)v(n-1) \quad (2.48)$$

¹This model is quite popular for the analysis of adaptive filters. We say more about this model in Sec. 3.2.1.

²Apparently, the definition of convergence in mean-square used for adaptive filtering analysis is different from the usual definition where $c = 0$.

Taking the expectation of both sides and using the above assumptions, we obtain

$$E[\Delta \mathbf{w}(n)] = (\mathbf{I} - \mu \mathbf{R}_u) E[\Delta \mathbf{w}(n-1)] \quad (2.49)$$

$$= \mathbf{X}(\mathbf{I} - \mu \mathbf{\Lambda})^n \mathbf{X}^T E[\Delta \mathbf{w}(0)] . \quad (2.50)$$

This recursion is the same as in the transient analysis of the steepest descent algorithm. We therefore have that the LMS algorithm is stable in the mean if

$$\boxed{0 < \mu < \frac{2}{\lambda_{\max}}} . \quad (2.51)$$

The bound given above ensures convergence in the mean, but places no constraint on the variance of $\Delta \mathbf{w}(n)$. Furthermore, since $\mathbf{R}_u$ is unknown, $\lambda_{\max}$ is unknown, and we have to estimate or upper bound it. We have that

$$\lambda_{\max} \leq \text{tr}(\mathbf{R}_u) = \sum_{m=1}^M r_u(0) = M r_u(0) = M E[u^2(n)] . \quad (2.52)$$

The expected power $E[u^2(n)]$ can be estimated as

$$\hat{E}[u^2(n)] = \frac{1}{M} \mathbf{u}^T(n) \mathbf{u}(n) . \quad (2.53)$$

Note that the estimation accuracy increases with the filter length M . From Parseval's Theorem, we also have that

$$E[u^2(n)] = \frac{1}{2\pi} \int_{-\pi}^{\pi} S_u(\omega) d\omega \leq \frac{1}{2\pi} \int_{-\pi}^{\pi} S_{\max} d\omega = S_{\max} \quad (2.54)$$

where $S_u(\omega)$ is the power spectral density of $u(n)$. Thus, we have that

$$\boxed{\frac{2}{M S_{\max}} \leq \frac{2}{\text{tr}(\mathbf{R}_u)} = \frac{2}{M E[u^2(n)]} \leq \frac{2}{\lambda_{\max}}} , \quad (2.55)$$

provided that the desired signal model is given by Eq. (2.45) and that $\mathbf{u}(n)$ is an IID random process.

Convergence in the Mean-Square

From Eq. (2.48), we have that

$$\begin{aligned} \|\Delta \mathbf{w}(n)\|^2 &= \| [\mathbf{I} - \mu \mathbf{u}(n-1) \mathbf{u}^T(n-1)] \Delta \mathbf{w}(n-1) - \mu \mathbf{u}(n-1) v(n-1) \|^2 \\ &= \| [\mathbf{I} - \mu \mathbf{u}(n-1) \mathbf{u}^T(n-1)] \Delta \mathbf{w}(n-1) \|^2 \\ &\quad + \mu^2 v^2(n-1) \|\mathbf{u}(n-1)\|^2 \\ &\quad - 2\mu v(n) \mathbf{u}^T(n-1) [\mathbf{I} - \mu \mathbf{u}(n-1) \mathbf{u}^T(n-1)] \Delta \mathbf{w}(n-1) \end{aligned} \quad (2.56)$$

Taking the expectation on both sides and using the above assumptions, we obtain

$$E[\|\Delta \mathbf{w}(n)\|^2] = E[\| [\mathbf{I} - \mu \mathbf{u}(n-1) \mathbf{u}^T(n-1)] \Delta \mathbf{w}(n-1) \|^2] + \mu^2 J_{\min} \text{tr}(\mathbf{\Lambda}) \quad (2.57)$$

where $J_{\min} = \sigma_v^2$. Evaluating the expected value of the first term and finding the values of the step-size μ for which $E[\|\Delta \mathbf{w}(n)\|^2]$ converges in the mean-square require a lot of work. In

Appendix B, we show how this can be done. Alternatively, a derivation can also be found in [7, pp. 452–465]. The final result is that the LMS filter is mean-square stable (and thus stable in the mean) if and only if the step-size satisfies [7, pp. 462]

$$f(\mu) = \frac{\mu}{2} \sum_{m=1}^M \frac{\lambda_m}{1 - \mu\lambda_m} = \frac{1}{2} \text{tr}(\mathbf{\Lambda}(\mu^{-1}\mathbf{I} - \mathbf{\Lambda})^{-1}) < 1 . \quad (2.58)$$

For small step-sizes $\mu \ll 1/\lambda_{\max}$, we have that

$$f(\mu) \approx \frac{\mu}{2} \sum_{m=1}^M \lambda_m = \frac{\mu}{2} \text{tr}(\mathbf{\Lambda}) < 1 . \quad (2.59)$$

which leads to that the LMS algorithm is mean-square stable if

$$0 < \mu < \frac{2}{\text{tr}(\mathbf{R}_u)} . \quad (2.60)$$

Learning Curve

The learning curve is also difficult to find. In Appendix B, we show that the learning curve is given by

$$J_1(\mathbf{w}(n)) = \mathbf{\Delta w}^T(0) \mathbf{X} \mathbf{S}(n) \mathbf{X}^T \mathbf{\Delta w}(0) + \mu^2 J_{\min} \sum_{i=0}^{n-1} \text{tr}(\mathbf{S}(i) \mathbf{\Lambda}) + J_{\min} \quad (2.61)$$

where

$$\mathbf{S}(n) = \mathbf{S}(n-1) - 2\mu \mathbf{\Lambda} \mathbf{S}(n-1) + \mu^2 [\mathbf{\Lambda} \text{tr}(\mathbf{S}(n-1) \mathbf{\Lambda}) + 2\mathbf{\Lambda} \mathbf{S}(n-1) \mathbf{\Lambda}] \quad (2.62)$$

with $\mathbf{S}(0) = \mathbf{\Lambda}$.

2.4.3 Steady-State Analysis

When the LMS algorithm is operating in steady-state and is mean-square stable, the EMSE, the mean-square deviation (MSD), and the misadjustment are [7, pp. 465]

$$J_{\text{ex}} = J_{\min} \frac{f(\mu)}{1 - f(\mu)} \quad (2.63)$$

$$\mathcal{M} = \frac{f(\mu)}{1 - f(\mu)} \quad (2.64)$$

$$E[\|\mathbf{\Delta w}(\infty)\|^2] = \frac{J_{\min}}{1 - f(\mu)} \frac{\mu}{2} \sum_{m=1}^M \frac{1}{1 - \mu\lambda_m} . \quad (2.65)$$

These results are also derived in Appendix B. For small step-sizes $\mu \ll 1/\lambda_{\max}$, the EMSE, MSD, and misadjustment simplify to [7, pp. 465]

$$J_{\text{ex}} \approx \mu J_{\min} \frac{\text{tr}(\mathbf{R}_u)}{2 - \mu \text{tr}(\mathbf{R}_u)} \approx \frac{\mu}{2} J_{\min} \text{tr}(\mathbf{R}_u) \quad (2.66)$$

$$\mathcal{M} \approx \mu \frac{\text{tr}(\mathbf{R}_u)}{2 - \mu \text{tr}(\mathbf{R}_u)} \approx \frac{\mu}{2} \text{tr}(\mathbf{R}_u) = \frac{\mu}{2} ME[u^2(n)] \quad (2.67)$$

$$E[\|\mathbf{\Delta w}(\infty)\|^2] \approx \mu J_{\min} \frac{M}{2 - \mu \text{tr}(\mathbf{R}_u)} \approx \frac{\mu}{2} J_{\min} M . \quad (2.68)$$

The approximated expression for the misadjustment shows that misadjustment is approximately proportional to the input power. This is undesirable and referred to as *gradient noise amplification*. In Lecture 3, we consider the normalised LMS algorithm which solves this problem.

2.4.4 Computational Cost

Table 2.1 shows the computational cost of the LMS algorithm in terms of the number of multiplications and additions or subtractions. From the table, we see that the total number of flops is $4M + 1$. Thus, the LMS algorithm has a linear complexity $\mathcal{O}(M)$ in the filter length M .

Term	$\times$	$+$ or $-$
$\mathbf{u}^T(n)\mathbf{w}(n)$	M	$M - 1$
$e(n) = d(n) - \mathbf{u}^T(n)\mathbf{w}(n)$		1
$\mu e(n)$	1	
$\mu \mathbf{u}(n)e(n)$	M	
$\mathbf{w}(n) + \mu \mathbf{u}(n)e(n)$		M
Total	$2M + 1$	$2M$

Table 2.1: Computational cost of the LMS algorithm.

Lecture 3

Normalised LMS and Affine Projection Algorithm

3.1 Review of the Basics

3.1.1 Inversion of a 2 by 2 Block Matrix

Let $\mathbf{A}$ and $\mathbf{M}_1 = \mathbf{D} - \mathbf{C}\mathbf{A}^{-1}\mathbf{B}$ be non-singular matrices. Then

$$\begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{bmatrix}^{-1} = \begin{bmatrix} \mathbf{A}^{-1} + \mathbf{A}^{-1}\mathbf{B}\mathbf{M}_1^{-1}\mathbf{C}\mathbf{A}^{-1} & -\mathbf{A}^{-1}\mathbf{B}\mathbf{M}_1^{-1} \\ -\mathbf{M}_1^{-1}\mathbf{C}\mathbf{A}^{-1} & \mathbf{M}_1^{-1} \end{bmatrix}. \quad (3.1)$$

Let instead $\mathbf{D}$ and $\mathbf{M}_2 = \mathbf{A} - \mathbf{B}\mathbf{D}^{-1}\mathbf{C}$ be non-singular matrices. Then

$$\begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{bmatrix}^{-1} = \begin{bmatrix} \mathbf{M}_2^{-1} & -\mathbf{M}_2^{-1}\mathbf{B}\mathbf{D}^{-1} \\ -\mathbf{D}^{-1}\mathbf{C}\mathbf{M}_2^{-1} & \mathbf{D}^{-1} + \mathbf{D}^{-1}\mathbf{C}\mathbf{M}_2^{-1}\mathbf{B}\mathbf{D}^{-1} \end{bmatrix}. \quad (3.2)$$

The matrices $\mathbf{M}_1$ and $\mathbf{M}_2$ are said to be the *Schur complement* of $\mathbf{A}$ and $\mathbf{D}$, respectively.

3.1.2 The Method of Lagrange Multipliers

The method of Lagrange multipliers can be used to solve optimisation problems with equality constraints¹

$$\begin{aligned} & \underset{\mathbf{x}}{\text{optimise}} && f(\mathbf{x}) \\ & \text{subject to} && \mathbf{h}(\mathbf{x}) = \mathbf{0} \end{aligned} \quad (3.3)$$

Consider the optimisation problem in Fig. 3.1 with the cost function $f(\mathbf{x})$ and the constraints $\mathbf{h}(\mathbf{x}) = \mathbf{0}$. At the constrained optimum $\mathbf{x}_c$, the gradients of $f(\mathbf{x})$ and $\mathbf{h}(\mathbf{x})$ are parallel. Thus,

$$\begin{aligned} \nabla f(\mathbf{x}) &= \lambda \nabla h(\mathbf{x}) && \text{(single constraint)} \\ \nabla f(\mathbf{x}) &= \sum_{m=1}^M \lambda_m \nabla h_m(\mathbf{x}) = \nabla h(\mathbf{x}) \boldsymbol{\lambda} && \text{(multiple constraints)} \end{aligned}$$

¹The method of Lagrange multipliers can also be used to solve the more general problem with inequality constraints, but we do not consider this here.

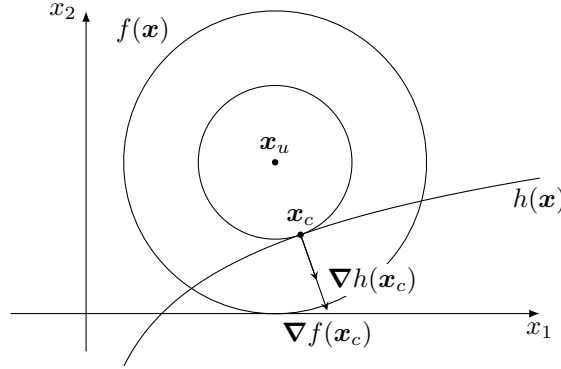


Figure 3.1: Optimisation problem with a single equality constraint. The points $\mathbf{x}_u$ and $\mathbf{x}_c$ are the unconstrained and the constrained minimum, respectively.

The vector $\boldsymbol{\lambda}$ contains the Lagrange multipliers, and they ensure that the gradients have the same direction and length.

If we define the Lagrangian function

$$L(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) - \boldsymbol{\lambda}^T \mathbf{h}(\mathbf{x}) , \quad (3.4)$$

then its critical points satisfy

$$\nabla_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{0} \quad \Longleftrightarrow \quad \nabla f(\mathbf{x}) = \nabla \mathbf{h}(\mathbf{x}) \boldsymbol{\lambda} \quad (3.5)$$

$$\nabla_{\boldsymbol{\lambda}} L(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{0} \quad \Longleftrightarrow \quad \mathbf{h}(\mathbf{x}) = \mathbf{0} . \quad (3.6)$$

Eq. (3.5) is the same as what we previously derived from Fig. 3.1, and Eq. (3.6) is simply the constraints of our optimisation problem. Thus, the solutions to the system of equations given by Eq. (3.5) and Eq. (3.6) are indeed the critical points of our optimisation problems. In order to find the optimum, we need to classify the critical points as either minimums, maximums, or saddle points. The classification of the critical points for any pair of cost function and constraints is beyond the scope of these lecture notes. We refer the interested reader to [3, pp. 294–308]. Alternatively, we may formulate the constrained optimisation problem in Eq. (3.3) as the sequential unconstrained optimisation problem [4, ch. 5]

$$\max_{\boldsymbol{\lambda}} \left\{ \underset{\mathbf{x}}{\text{optimise}} L(\mathbf{x}, \boldsymbol{\lambda}) \right\} . \quad (3.7)$$

We could solve this optimisation problem by applying the five step recipe for unconstrained optimisation in Sec. 1.1.2 to first the inner optimisation problem and then the outer optimisation problem.

We are here not concerned with these general methods. Instead, we consider the simpler problem of minimising a quadratic cost function with linear equality constraints.

Quadratic Cost Function with Linear Equality Constraints

We consider the following equality constrained minimisation problem

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{P} \mathbf{x} + \mathbf{q}^T \mathbf{x} + r \\ \text{subject to} \quad & \mathbf{h}(\mathbf{x}) = \mathbf{A} \mathbf{x} - \mathbf{b} = \mathbf{0} \end{aligned} \quad (3.8)$$

in which the cost function is quadratic and the equality constraints are linear. We assume that $\mathbf{P}$ is an $N \times N$ p.d. matrix so that $f(\mathbf{x})$ is a convex function. Moreover, we assume that $\mathbf{A}$ is a full rank $M \times N$ matrix with $M \leq N$. The Langrangian function is

$$L(\mathbf{x}, \boldsymbol{\lambda}) = \frac{1}{2} \mathbf{x}^T \mathbf{P} \mathbf{x} + \mathbf{q}^T \mathbf{x} + r - \boldsymbol{\lambda}^T (\mathbf{A} \mathbf{x} - \mathbf{b}) \quad (3.9)$$

and its gradient w.r.t. $\mathbf{x}$ is

$$\nabla_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{P} \mathbf{x} + \mathbf{q} - \mathbf{A}^T \boldsymbol{\lambda} . \quad (3.10)$$

The system of equations given by Eq. (3.5) and Eq. (3.6) are therefore

$$\begin{bmatrix} \mathbf{P} & -\mathbf{A}^T \\ -\mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} -\mathbf{q} \\ -\mathbf{b} \end{bmatrix} . \quad (3.11)$$

Since the cost function $f(\mathbf{x})$ is assumed to be convex, there is only one point satisfying this system of equations. By using the 2×2 block matrix inversion rule in Eq. (3.1), we obtain that this point $(\mathbf{x}_c, \boldsymbol{\lambda}_c)$ is

$$\boldsymbol{\lambda}_c = (\mathbf{A} \mathbf{P}^{-1} \mathbf{A}^T)^{-1} (\mathbf{A} \mathbf{P}^{-1} \mathbf{q} + \mathbf{b}) \quad (3.12)$$

$$\mathbf{x}_c = \mathbf{P}^{-1} (\mathbf{A}^T \boldsymbol{\lambda}_c - \mathbf{q}) . \quad (3.13)$$

Thus, the solution to the constrained minimisation problem in Eq. (3.8) is $\mathbf{x}_c$ given by Eq. (3.13).

3.2 Overview over Adaptive Filters based on the Mean-Squared Error Cost Function

An overview over the adaptive filters is shown in Fig. A.1. Recall that we have the following important properties for the steepest descent algorithm and the stochastic gradient methods.

Steepest Descent (SD)

- Since the statistics is assumed known, the gradient and filter coefficients are deterministic functions.
- SD is of limited practical usage, but illustrates nicely how adaptive filters function and how they are analysed.
- SD is the "ideal" (first-order) adaptive filter.

Stochastic Gradient Methods (SGMs)

- Since the statistics is unknown, the gradient and filter coefficients are stochastic processes.
- They are generally easy to implement and hard to analyse.
- They are approximations to the steepest descent algorithm.

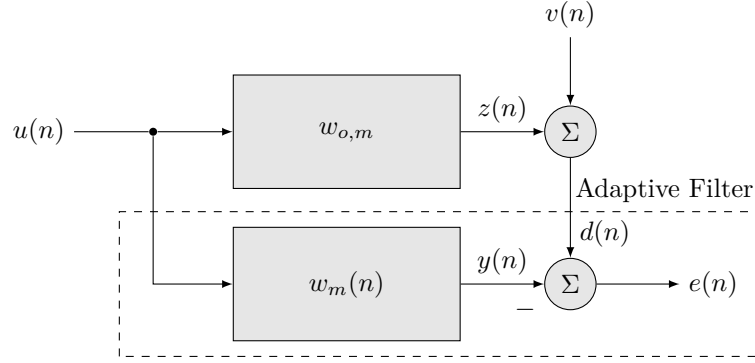


Figure 3.2: Block diagram showing the model used to analyse adaptive filters.

3.2.1 Model for the Analysis of SGMs

In order to make the SGMs easier to analyse, a model is typically assumed for the relationship between the input signal $u(n)$ and the desired signal $d(n)$. Fig. 3.2 shows one of the most popular models used for the analysis of adaptive filters. Define

$$\mathbf{w}_o = [w_{o,0} \ \cdots \ w_{o,M-1}]^T \quad (3.14)$$

$$\mathbf{w}(n) = [w_0(n) \ \cdots \ w_{M-1}(n)]^T \quad (3.15)$$

$$\mathbf{u}(n) = [u(n) \ \cdots \ u(n-M+1)]^T \quad (3.16)$$

$$\Delta \mathbf{w}(n) = \mathbf{w}_o - \mathbf{w}(n) \quad (3.17)$$

where $\Delta \mathbf{w}(n)$ is referred to as the weight error.

Assumptions

From Fig. 3.2, we have that

$$d(n) = z(n) + v(n) = \mathbf{u}^T(n) \mathbf{w}_o + v(n) . \quad (3.18)$$

Moreover, we assume that

1. $v(n)$ is white noise with variance σ_v^2 .
2. $v(n)$ and $u(n)$ are uncorrelated.
3. $\mathbf{u}(n)$ is a white process with correlation matrix $\mathbf{R}_u$. This assumption implies that $\mathbf{u}(n)$ and $\mathbf{w}(n)$ are uncorrelated.

Equations

We have that

$$e(n) = d(n) - y(n) = v(n) + \mathbf{u}^T(n) \mathbf{w}_o - \mathbf{u}^T(n) \mathbf{w}(n) \quad (3.19)$$

$$= \mathbf{u}^T(n) \Delta \mathbf{w}(n) + v(n) \quad (3.20)$$

Thus, at the optimum where $\Delta \mathbf{w}(n) = \mathbf{0}$, we have that

$$J_1(\mathbf{w}_o) = J_{\min} = J_1(v^2(n)) = \sigma_v^2 . \quad (3.21)$$

The cost function can be written as

$$\begin{aligned} J_1(\mathbf{w}(n)) &= E[e^2(n)] = E[v^2(n)] + E[\Delta \mathbf{w}^T(n) \mathbf{u}(n) \mathbf{u}^T(n) \Delta \mathbf{w}(n)] \\ &= \sigma_v^2 + E[E[\Delta \mathbf{w}^T(n) \mathbf{u}(n) \mathbf{u}^T(n) \Delta \mathbf{w}(n) | \Delta \mathbf{w}(n)]] \\ &= J_{\min} + E[\Delta \mathbf{w}^T(n) \mathbf{R}_u \Delta \mathbf{w}(n)] . \end{aligned} \quad (3.22)$$

When viewed as a function of n , the cost function is called the learning curve.

3.2.2 How to Analyse Adaptive Filters

Adaptive filters can be analysed in several ways.

Transient Performance How does the filter handle an abrupt change in the statistics of $u(n)$, and how fast does it converge to steady-state?

Steady-state Performance How does the filter perform in a WSS environment after all transients have died out?

Tracking Performance How does the filter handle slow variations in the statistics of $u(n)$ and/or $d(n)$?

Numerical Precision Effects What happens when the filter is implemented on a finite precision computer?

Computational Complexity How much processing time does the algorithm require?

Important Values and Functions

When analysing adaptive filters, we typically quantify their performance in terms of the following values and functions.

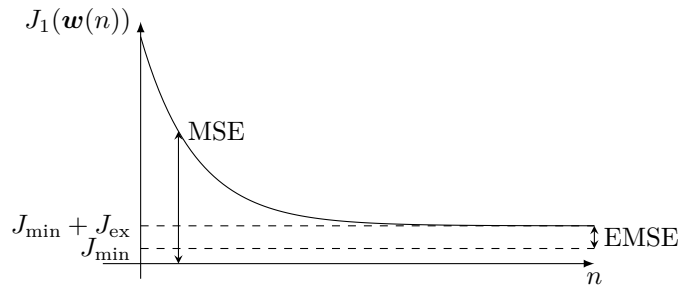


Figure 3.3: The cost function (same as the MSE), the minimum MSE, and the excess MSE.

$$J_1(\mathbf{w}(n)) \quad (\text{Mean-Square Error (MSE)}) \quad (3.23)$$

$$J_1(\mathbf{w}_o) = J_{\min} \quad (\text{Minimum MSE (MMSE)}) \quad (3.24)$$

$$J_{\text{ex}} = J_1(\mathbf{w}(\infty)) - J_{\min} \quad (\text{Excess MSE (EMSE)}) \quad (3.25)$$

$$\mathcal{M} = \frac{J_{\text{ex}}}{J_{\min}} \quad (\text{Misadjustment}) \quad (3.26)$$

$$E[\|\Delta\mathbf{w}(\infty)\|^2] \quad (\text{Mean-Square Deviation (MSD)}) \quad (3.27)$$

$$\lambda_{\max} \quad (\text{Maximum Eigenvalue of } \mathbf{R}_u) \quad (3.28)$$

$$\lambda_{\min} \quad (\text{Minimum Eigenvalue of } \mathbf{R}_u) \quad (3.29)$$

$$\kappa(\mathbf{R}_u) = \frac{\lambda_{\max}}{\lambda_{\min}} \quad (\text{Condition number of } \mathbf{R}_u) \quad (3.30)$$

In adaptive filtering, some people use the term eigenvalue spread $\chi(\mathbf{R}_u)$ instead of the condition number. The MSE, the EMSE, the MSD, and the misadjustment depend on the adaptive filtering algorithm. Conversely, the remaining parameters depend on the desired signal and the input signal and are thus independent of the adaptive filtering algorithm. In Fig. 3.3, some of the quantities are shown.

3.3 LMS Revisited

Recall the steepest descent algorithm

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \frac{\mu}{2} \mathbf{g}(\mathbf{w}(n)) \quad (3.31)$$

where

$$\mathbf{g}(\mathbf{w}(n)) = 2\mathbf{R}_u\mathbf{w}(n) - 2\mathbf{r}_{ud} = -2E[\mathbf{u}(n)e(n)] \quad (3.32)$$

is the gradient of $J_1(\mathbf{w}(n))$. Replacing the gradient with the simple estimate

$$\hat{\mathbf{g}}(\mathbf{w}(n)) = -2\mathbf{u}(n)e(n) \quad (3.33)$$

leads to the LMS algorithm

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu\mathbf{u}(n)e(n) \quad (3.34)$$

$$= (\mathbf{I} - \mu\mathbf{u}(n)\mathbf{u}^T(n))\mathbf{w}(n) + \mu\mathbf{u}(n)d(n) . \quad (3.35)$$

In order to analyse the LMS algorithm, we use the analysis model in Sec. 3.2.1.

3.3.1 Transient Analysis

We would like that

$$\lim_{n \rightarrow \infty} E[\Delta\mathbf{w}(n)] = 0 \quad (\text{Convergence in mean})$$

$$\lim_{n \rightarrow \infty} E[\|\Delta\mathbf{w}(n)\|^2] = c < \infty \quad (\text{Convergence in mean-square})$$

Convergence in the Mean

We have that

$$\Delta \mathbf{w}(n+1) = (\mathbf{I} - \mu \mathbf{u}(n) \mathbf{u}^T(n)) \Delta \mathbf{w}(n) + \mu \mathbf{u}(n) v(n) \quad (3.36)$$

from which we get

$$E[\Delta \mathbf{w}(n+1)] = (\mathbf{I} - \mu \mathbf{R}_u) E[\Delta \mathbf{w}(n)] . \quad (3.37)$$

Thus, the LMS algorithm is stable in the mean if (same as for SD)

$$0 < \mu < \frac{2}{\lambda_{\max}} . \quad (3.38)$$

Since $\lambda_{\max}$ is unknown and hard to estimate, we can bound it by

$$\lambda_{\max} \leq \text{tr}(\mathbf{R}_u) = ME[u^2(n)] \leq MS_{\max} . \quad (3.39)$$

Estimating the power $E[u^2(n)]$ or the maximum is the power spectral density $S_{\max}$ is fairly easy.

Convergence in the Mean-Square

We assume that $\mathbf{u}(n)$ has a Gaussian distribution with zero-mean and covariance matrix $\mathbf{R}_u$. Then, as we have shown in Appendix B, we get that the LMS algorithm is stable in the mean-square if the step-size satisfies

$$f(\mu) = \frac{\mu}{2} \sum_{m=1}^M \frac{\lambda_m}{1 - \mu \lambda_m} < 1 . \quad (3.40)$$

For small step-sizes $\mu \ll 1/\lambda_{\max}$, we have

$$f(\mu) \approx \frac{\mu}{2} \sum_{m=1}^M \lambda_m = \frac{\mu}{2} \text{tr}(\mathbf{R}_u) < 1 . \quad (3.41)$$

Thus, stability in the mean-square requires that the step-size satisfies

$$0 < \mu < \frac{2}{\text{tr}(\mathbf{R}_u)} . \quad (3.42)$$

3.3.2 Steady-State Analysis

From Appendix B, we have that

$$\text{EMSE:} \quad J_{\text{ex}} = J_1(\mathbf{w}(\infty)) - J_{\min} = J_{\min} \frac{f(\mu)}{1 - f(\mu)} \quad (3.43)$$

$$\approx \frac{\mu}{2} J_{\min} \text{tr}(\mathbf{R}_u) \quad (3.44)$$

$$\text{Misadjustment:} \quad \mathcal{M} = \frac{J_{\text{ex}}}{J_{\min}} = \frac{f(\mu)}{1 - f(\mu)} \quad (3.45)$$

$$\approx \frac{\mu}{2} \text{tr}(\mathbf{R}_u) \quad (3.46)$$

$$\text{MSD:} \quad E[\|\Delta \mathbf{w}(\infty)\|^2] \approx \frac{\mu}{2} J_{\min} M \quad (3.47)$$

Here, the approximations are valid for small step-sizes. We see that the EMSE is approximately proportional to the input power. This is a problem and referred to as *gradient noise amplification*.

3.4 Normalised LMS Adaptive Filters

Recall that we can bound the maximum eigenvalue by

$$\lambda_{\max} \leq \text{tr}(\mathbf{R}_u) = ME[u^2(n)] . \quad (3.48)$$

An estimate of the power could be

$$\hat{E}[u^2(n)] = \frac{1}{M} \mathbf{u}^T(n) \mathbf{u}(n) = \frac{1}{M} \|\mathbf{u}(n)\|^2 . \quad (3.49)$$

This leads to the following bound on the step-size

$$0 < \mu < \frac{2}{\|\mathbf{u}(n)\|^2} \iff 0 < \mu \|\mathbf{u}(n)\|^2 < 2 . \quad (3.50)$$

We may define the time-varying step-size

$$\mu(n) = \frac{\beta}{\|\mathbf{u}(n)\|^2} , \quad 0 < \beta < 2 . \quad (3.51)$$

Then, the LMS algorithm is

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\beta}{\|\mathbf{u}(n)\|^2} \mathbf{u}(n) e(n) \quad (3.52)$$

which is called the normalised LMS (NLMS) algorithm. The NLMS bypasses the problem of gradient noise amplification by normalising the gradient estimate by the input power. However, this normalisation causes numerical problems when the input power is close to zero. In order to avoid this, we introduce the regularisation parameter ϵ , which is a small positive constant, into the NLMS algorithm

$$\boxed{\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\beta}{\epsilon + \|\mathbf{u}(n)\|^2} \mathbf{u}(n) e(n)} . \quad (3.53)$$

Fast Computation

The power estimate may be computed very efficiently by writing it in a recursive manner

$$\begin{aligned} \|\mathbf{u}(n)\|^2 &= \sum_{m=0}^{M-1} u^2(n-m) = u^2(n) - u^2(n-M) + \sum_{m=1}^M u^2(n-m) \\ &= u^2(n) - u^2(n-M) + \|\mathbf{u}(n-1)\|^2 . \end{aligned} \quad (3.54)$$

It should be noted that the use of this recursion can be problematic in practice due to accumulated rounding errors [7, p. 227]. These rounding errors may potentially cause the norm to be negative.

3.4.1 Transient Analysis

In order to analyse the NLMS algorithm, we use the analysis model in Sec. 3.2.1. If $\sigma_v^2 = 0$, the NLMS algorithm converges in the mean and the mean-square if [8, p. 325]

$$\boxed{0 < \beta < 2} . \quad (3.55)$$

Moreover, the optimal step-size is

$$\beta_o = 1 . \quad (3.56)$$

If $\sigma_v^2 > 0$, the optimal step-size is more complicated and can be found in [8, p. 325].

3.4.2 Steady-State Analysis

Assuming the analysis model in Sec. 3.2.1, it can be shown that [7, pp. 300–302, p. 474]

$$\begin{aligned} \text{EMSE:} \quad J_{\text{ex}} &= J_1(\mathbf{w}(\infty)) - J_{\min} \\ &\approx \frac{\beta}{2} J_{\min} \text{tr}(\mathbf{R}_u) E \left[\frac{1}{\|\mathbf{u}(n)\|^2} \right] \geq \frac{\beta}{2} J_{\min} \end{aligned} \quad (3.57)$$

$$\text{Misadjustment:} \quad \mathcal{M} = \frac{J_{\text{ex}}}{J_{\min}} \approx \frac{\beta}{2} \text{tr}(\mathbf{R}_u) E \left[\frac{1}{\|\mathbf{u}(n)\|^2} \right] \geq \frac{\beta}{2} \quad (3.58)$$

$$\text{MSD:} \quad E[\|\Delta \mathbf{w}(\infty)\|^2] \approx \frac{\beta}{2} J_{\min} E \left[\frac{1}{\|\mathbf{u}(n)\|^2} \right] \geq \frac{\beta J_{\min}}{2 \text{tr}(\mathbf{R}_u)} \quad (3.59)$$

where the inequality follows from that $E[x^{-1}] \geq E[x]^{-1}$. The approximations are valid for small values of β and ϵ . Note that the approximation of the misadjustment no longer depends on the input power. This is a consequence of the normalisation of the LMS gradient with $\|\mathbf{u}(n)\|^2$. Moreover, it should be noted that there exist several other approximations to the EMSE and the MSD than presented above [7, p. 474].

3.4.3 Computational Cost

Table 3.1 shows the computational cost of the NLMS algorithm in terms of the number of multiplications, additions or subtractions, and divisions. From the table, we see that the total number of flops is $6M + 2$. If we use the fast computation of the normalisation constant, the computational cost is reduced to $4M + 7$ flops as shown in Table 3.2. Thus, the NLMS algorithm has a linear complexity in the filter length $\mathcal{O}(M)$ for both versions of the NLMS algorithm.

Term	$\times$	$+$ or $-$	$/$
$\ \mathbf{u}(n)\ ^2$	M	$M - 1$	
$\mathbf{u}^T(n)\mathbf{w}(n)$	M	$M - 1$	
$e(n) = d(n) - \mathbf{u}^T(n)\mathbf{w}(n)$		1	
$\beta e(n)/(\epsilon + \ \mathbf{u}(n)\ ^2)$	1	1	1
$\beta \mathbf{u}(n)e(n)/(\epsilon + \ \mathbf{u}(n)\ ^2)$	M		
$\mathbf{w}(n) + \beta \mathbf{u}(n)e(n)/(\epsilon + \ \mathbf{u}(n)\ ^2)$		M	
Total	$3M + 1$	$3M$	1

Table 3.1: Computational cost of the NLMS algorithm.

3.4.4 Another Derivation of the NLMS Algorithm

We have two requirements to the adaptive filter.

1. If we filter the input sample at time n through the filter at time $n + 1$, the error should be zero. That is,

$$d(n) - \mathbf{u}^T(n)\mathbf{w}(n+1) = 0. \quad (3.60)$$

There are an infinite number of solutions that satisfy this requirement if $M > 1$.

2. Among all the filter coefficient vectors $\mathbf{w}(n+1)$ satisfying the first requirements, we select the vector resulting in the smallest change from iteration n to $n + 1$. That is, $\|\mathbf{w}(n+1) - \mathbf{w}(n)\|^2$ should be as small as possible.

Term	$\times$	$+$ or $-$	$/$
$\ \mathbf{u}(n)\ ^2$	2	2	
$\mathbf{u}^T(n)\mathbf{w}(n)$	M	$M - 1$	
$e(n) = d(n) - \mathbf{u}^T(n)\mathbf{w}(n)$		1	
$\beta e(n)/(\epsilon + \ \mathbf{u}(n)\ ^2)$	1	1	1
$\beta \mathbf{u}(n)e(n)/(\epsilon + \ \mathbf{u}(n)\ ^2)$	M		
$\mathbf{w}(n) + \beta \mathbf{u}(n)e(n)/(\epsilon + \ \mathbf{u}(n)\ ^2)$		M	
Total	$2M + 3$	$2M + 3$	1

Table 3.2: Computational cost of the NLMS algorithm with fast update of the normalisation factor.

These two requirements lead to the following constrained minimisation problem

$$\begin{aligned} \min_{\mathbf{w}(n+1)} \quad & f(\mathbf{w}(n+1)) = \|\mathbf{w}(n+1) - \mathbf{w}(n)\|^2 \\ \text{s.t.} \quad & h(\mathbf{w}(n+1)) = d(n) - \mathbf{u}^T(n)\mathbf{w}(n+1) = 0 \end{aligned} \quad (3.61)$$

This optimisation problem has a quadratic cost function and a single linear equality constraint, and it is therefore on the same form as Eq. (3.8) with

$$\mathbf{x} = \mathbf{w}(n+1) \quad (3.62)$$

$$\mathbf{A} = -\mathbf{u}^T(n) \quad (3.63)$$

$$\mathbf{b} = -d(n) \quad (3.64)$$

$$\mathbf{P} = 2\mathbf{I} \quad (3.65)$$

$$\mathbf{q} = -2\mathbf{w}(n) \quad (3.66)$$

$$\mathbf{r} = \|\mathbf{w}(n)\|^2. \quad (3.67)$$

Thus, from Eq. (3.12) and Eq. (3.13), we readily obtain the solution to the optimisation problem in Eq. (3.61). The solution is given by

$$\lambda = -\frac{2}{\|\mathbf{u}(n)\|^2}e(n) \quad (3.68)$$

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{1}{\|\mathbf{u}(n)\|^2}\mathbf{u}(n)e(n). \quad (3.69)$$

The last equation is identical to the NLMS algorithm with $\beta = 1$ and $\epsilon = 0$.

3.5 Affine Projection Adaptive Filters

We may extend the requirements to the adaptive filter in the following way.

1. If we filter the input sample at time $n - k$ for $k = 0, 1, \dots, K - 1 < M$ through the filter at time $n + 1$, the corresponding errors should be zero. That is,

$$d(n - k) - \mathbf{u}^T(n - k)\mathbf{w}(n + 1) = 0, \quad k = 0, 1, \dots, K - 1 < M. \quad (3.70)$$

If we define

$$\mathbf{U}(n) = [\mathbf{u}(n) \quad \mathbf{u}(n - 1) \quad \dots \quad \mathbf{u}(n - K + 1)] \quad (3.71)$$

$$\mathbf{d}(n) = [d(n) \quad d(n - 1) \quad \dots \quad d(n - K + 1)]^T, \quad (3.72)$$

we may write

$$\mathbf{d}(n) - \mathbf{U}^T(n)\mathbf{w}(n+1) = \mathbf{0} . \quad (3.73)$$

If $K = M$ and $\mathbf{U}(n)$ has full rank, there is only one filter coefficient vector $\mathbf{w}(n+1)$ satisfying this requirement. It is given by

$$\mathbf{w}(n+1) = \mathbf{U}^{-T}(n)\mathbf{d}(n) . \quad (3.74)$$

When this is not the case, there are an infinite number of solutions that satisfy this requirement, and we therefore need a second requirement.

2. Among all the filter coefficient vectors $\mathbf{w}(n+1)$ satisfying the first requirements, we select the vector resulting in the smallest change from iteration n to $n+1$. That is, $\|\mathbf{w}(n+1) - \mathbf{w}(n)\|^2$ should be as small as possible.

Note that for $K = 1$, the requirements are identical to the requirements leading to the NLMS algorithm. The affine projections algorithm (APA) can therefore be viewed as a generalisation of the NLMS algorithm. We may formulate the requirements as a constrained optimisation problem

$$\begin{aligned} \min_{\mathbf{w}(n+1)} \quad & f(\mathbf{w}(n+1)) = \|\mathbf{w}(n+1) - \mathbf{w}(n)\|^2 \\ \text{s.t.} \quad & \mathbf{h}(\mathbf{w}(n+1)) = \mathbf{d}(n) - \mathbf{U}^T(n)\mathbf{w}(n+1) = \mathbf{0} \end{aligned} \quad (3.75)$$

This optimisation problem has a quadratic cost function and $K \leq M$ linear equality constraints, and it is therefore on the same form as Eq. (3.8) with

$$\mathbf{x} = \mathbf{w}(n+1) \quad (3.76)$$

$$\mathbf{A} = -\mathbf{U}^T(n) \quad (3.77)$$

$$\mathbf{b} = -\mathbf{d}(n) \quad (3.78)$$

$$\mathbf{P} = 2\mathbf{I} \quad (3.79)$$

$$\mathbf{q} = -2\mathbf{w}(n) \quad (3.80)$$

$$\mathbf{r} = \|\mathbf{w}(n)\|^2 . \quad (3.81)$$

Thus, from Eq. (3.12) and Eq. (3.13), we readily obtain the solution to the optimisation problem in Eq. (3.75). The solution is given by

$$\boldsymbol{\lambda} = -2 \left(\mathbf{U}^T(n)\mathbf{U}(n) \right)^{-1} \mathbf{e}(n) \quad (3.82)$$

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mathbf{U}(n) \left(\mathbf{U}^T(n)\mathbf{U}(n) \right)^{-1} \mathbf{e}(n) . \quad (3.83)$$

From this result, we see that the APA reduces to the NLMS algorithm for $K = 1$. Usually, we add a regularisation parameter ϵ and a step-size parameter β to the algorithm and obtain

$$\boxed{\mathbf{w}(n+1) = \mathbf{w}(n) + \beta \mathbf{U}(n) \left(\epsilon \mathbf{I} + \mathbf{U}^T(n)\mathbf{U}(n) \right)^{-1} \mathbf{e}(n)} . \quad (3.84)$$

As for the NLMS algorithm, ϵ is a small positive value which bypasses numerical problems when $\mathbf{U}^T(n)\mathbf{U}(n)$ is ill-conditioned.

For $K > 1$, The computational complexity of the APA is higher than that of the NLMS algorithm since we have to invert a $K \times K$ matrix. Although there exist fast ways of doing this [8, pp. 339–340], the APA is more expensive than the NLMS algorithm.

3.5.1 Transient Analysis

In order to analyse the APA, we use the analysis model in Sec. 3.2.1. If $\sigma_v^2 = 0$, the APA converges in the mean and the mean-square if [8, p. 337]

$$\boxed{0 < \beta < 2} . \quad (3.85)$$

Moreover, the optimal step-size is

$$\beta_o = 1 . \quad (3.86)$$

If $\sigma_v^2 > 0$, the optimal step-size is more complicated and can be found in [8, p. 337].

3.5.2 Steady-State Analysis

Assuming the analysis model in Sec. 3.2.1, it can be shown that [7, p. 327]

$$\begin{aligned} \text{EMSE:} \quad J_{\text{ex}} &= J_1(\mathbf{w}(\infty)) - J_{\min} \\ &\approx \frac{\beta}{2} J_{\min} \text{tr}(\mathbf{R}_u) E \left[\frac{K}{\|\mathbf{u}(n)\|^2} \right] \geq \frac{\beta}{2} J_{\min} K \end{aligned} \quad (3.87)$$

$$\text{Misadjustment:} \quad \mathcal{M} = \frac{J_{\text{ex}}}{J_{\min}} \approx \frac{\beta}{2} \text{tr}(\mathbf{R}_u) E \left[\frac{K}{\|\mathbf{u}(n)\|^2} \right] \geq \frac{\beta K}{2} \quad (3.88)$$

where the inequality follows from that $E[x^{-1}] \geq E[x]^{-1}$. The approximations are valid for small values of β and ϵ . It should be noted that there exist several other approximations to the EMSE than presented above [7, p. 325]. For more accurate expressions and for an expression for the mean-square deviation, see [7, pp. 510–512] and [9].

3.5.3 Computational Cost

Table 3.3 shows the computational cost of the APA algorithm in terms of the number of multiplications and additions or subtractions. We have assumed that the cost of inverting a $K \times K$ matrix is K^3 multiplications and additions [7, p. 240]. From the table, we see that the total number of flops is $2M(K^2 + 2K) + 2K^3 + K^2 + M$. Thus, the APA has a complexity of $\mathcal{O}(MK^2)$. Note that there exist faster ways of implementing the APA [8, pp. 339–340].

Term	$\times$	$+$ or $-$
$\mathbf{U}^T(n)\mathbf{w}(n)$	MK	$(M-1)K$
$\mathbf{e}(n) = \mathbf{d}(n) - \mathbf{U}^T(n)\mathbf{w}(n)$		K
$\mathbf{U}^T(n)\mathbf{U}(n)$	MK^2	$(M-1)K^2$
$\epsilon\mathbf{I} + \mathbf{U}^T(n)\mathbf{U}(n)$		K
$(\epsilon\mathbf{I} + \mathbf{U}^T(n)\mathbf{U}(n))^{-1}$	K^3	K^3
$(\epsilon\mathbf{I} + \mathbf{U}^T(n)\mathbf{U}(n))^{-1}\mathbf{e}(n)$	K^2	$K(K-1)$
$\mathbf{U}(n)(\epsilon\mathbf{I} + \mathbf{U}^T(n)\mathbf{U}(n))^{-1}\mathbf{e}(n)$	MK	$M(K-1)$
$\beta\mathbf{U}(n)(\epsilon\mathbf{I} + \mathbf{U}^T(n)\mathbf{U}(n))^{-1}\mathbf{e}(n)$	M	
$\mathbf{w}(n) + \beta\mathbf{U}(n)(\epsilon\mathbf{I} + \mathbf{U}^T(n)\mathbf{U}(n))^{-1}\mathbf{e}(n)$		M
Total	$M(K^2 + 2K + 1) + K^3 + K^2$	$M(K^2 + 2K) + K^3$

Table 3.3: Computational cost of the APA.

Lecture 4

Recursive Least-Squares Adaptive Filters

4.1 Review of the Basics

4.1.1 The Matrix Inversion Lemma

Let $\mathbf{X}$, $\mathbf{Y}$, $\mathbf{X} + \mathbf{U}\mathbf{Y}\mathbf{V}$, and $\mathbf{Y}^{-1} + \mathbf{V}\mathbf{X}^{-1}\mathbf{U}$ all be non-singular matrices. By equating element (1,1) of the two block matrices in Eq. (3.1) and Eq. (3.2) and setting

$$\mathbf{X} = \mathbf{A} \tag{4.1}$$

$$\mathbf{U} = \mathbf{B} \tag{4.2}$$

$$\mathbf{V} = \mathbf{C} \tag{4.3}$$

$$\mathbf{Y} = -\mathbf{D}^{-1}, \tag{4.4}$$

we obtain the matrix inversion lemma

$$(\mathbf{X} + \mathbf{U}\mathbf{Y}\mathbf{V})^{-1} = \mathbf{X}^{-1} - \mathbf{X}^{-1}\mathbf{U}(\mathbf{Y}^{-1} + \mathbf{V}\mathbf{X}^{-1}\mathbf{U})^{-1}\mathbf{V}\mathbf{X}^{-1}. \tag{4.5}$$

The matrix inversion lemma is also sometimes called the *Woodbury matrix identity* or the *Woodbury's identity*.

4.2 Method of Least-Squares

The adaptive filtering problem is shown in Fig. 4.1. From the figure, we have that

$u(n)$: zero-mean input signal

$w_m(n)$: M -tap FIR-filter with impulse response $w_0(n), w_1(n), \dots, w_{M-1}(n)$

$y(n)$: output signal given by $y(n) = \sum_{m=0}^{M-1} w_m(n)u(n-m)$

$d(n)$: zero-mean desired signal

$e(n)$: error signal

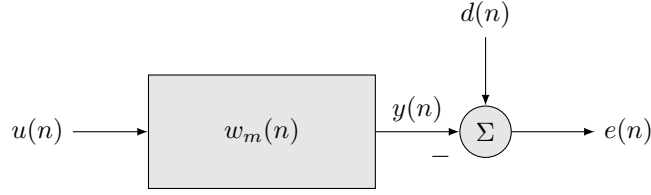


Figure 4.1: Block diagram of adaptive filtering in a non-WSS environment.

Define

$$\mathbf{w}(n) = [w_0(n) \quad w_1(n) \quad \cdots \quad w_{M-1}(n)]^T \quad (4.6)$$

$$\mathbf{u}(n) = [u(n) \quad u(n-1) \quad \cdots \quad u(n-M+1)]^T. \quad (4.7)$$

For $i = 1, 2, \dots, n$ with $n \geq M$, define

$$\mathbf{A}(n) = [\mathbf{u}(1) \quad \mathbf{u}(2) \quad \cdots \quad \mathbf{u}(n)]^T \quad (4.8)$$

$$\mathbf{d}(n) = [d(1) \quad d(2) \quad \cdots \quad d(n)]^T \quad (4.9)$$

$$\mathbf{e}(n) = [e(1) \quad e(2) \quad \cdots \quad e(n)]^T. \quad (4.10)$$

Then

$$\mathbf{e}(n) = \mathbf{d}(n) - \mathbf{A}(n)\mathbf{w}(n) \iff \mathbf{d}(n) = \mathbf{A}(n)\mathbf{w}(n) + \mathbf{e}(n). \quad (4.11)$$

Note that we have made $\mathbf{A}(n)$, $\mathbf{d}(n)$, and $\mathbf{e}(n)$ time-dependent in order to emphasise that we are here concerned with an online algorithm. Therefore, we also formulate the squared error cost function $J_2(\mathbf{w}(n))$ in a slightly different way compared to Eq. (1.29). Here, we define it as

$$J_2(\mathbf{w}(n)) = \sum_{i=1}^n e^2(i) = \mathbf{e}^T(n)\mathbf{e}(n). \quad (4.12)$$

In the method of least-squares, we wish to minimise $J_2(\mathbf{w}(n))$ which we can write as

$$\begin{aligned} J_2(\mathbf{w}(n)) &= \mathbf{e}^T(n)\mathbf{e}(n) = (\mathbf{d}(n) - \mathbf{A}(n)\mathbf{w}(n))^T(\mathbf{d}(n) - \mathbf{A}(n)\mathbf{w}(n)) \\ &= \mathbf{d}^T(n)\mathbf{d}(n) + \mathbf{w}^T(n)\mathbf{A}^T(n)\mathbf{A}(n)\mathbf{w}(n) - 2\mathbf{w}^T(n)\mathbf{A}^T(n)\mathbf{d}(n). \end{aligned} \quad (4.13)$$

The minimiser

$$\mathbf{w}_o(n) = (\mathbf{A}^T(n)\mathbf{A}(n))^{-1}\mathbf{A}^T(n)\mathbf{d}(n) \quad (4.14)$$

is referred to as the least-squares solution, and it is the unique solution to the normal equations

$$\mathbf{A}^T(n)\mathbf{A}(n)\mathbf{w}(n) = \mathbf{A}^T(n)\mathbf{d}(n), \quad (4.15)$$

provided that $\mathbf{A}(n)$ has full rank.

4.2.1 Weighted Least-Squares

When the statistics of $u(n)$ and/or $d(n)$ is time dependent, the minimisation of the squared error $J_2(\mathbf{w}(n))$ may fail to give a good estimate at time n since all data affect the value of $J_2(\mathbf{w}(n))$

with the same weight. Ideally, we would like that the new data are assigned a larger weight than the old data. In order to do this, we reformulate $J_2(\mathbf{w}(n))$ as a weighted cost function

$$J_\beta(\mathbf{w}(n)) = \sum_{i=1}^n \beta(n, i) e^2(i) = \mathbf{e}^T(n) \mathbf{B}(n) \mathbf{e}(n) \quad (4.16)$$

where $\beta(n, i)$ contains the weight pertaining to the i 'th error at time n , and $\mathbf{B}(n)$ is a diagonal matrix given by

$$\mathbf{B}(n) = \text{diag}(\beta(n, 1), \beta(n, 2), \dots, \beta(n, n)) . \quad (4.17)$$

We use the five step recipe in Sec. 1.1.2 to minimise $J_\beta(\mathbf{w}(n))$ w.r.t $\mathbf{w}(n)$.

1. Construct the cost function

$$\begin{aligned} J_\beta(\mathbf{w}(n)) &= \mathbf{e}^T(n) \mathbf{B}(n) \mathbf{e}(n) = (\mathbf{d}(n) - \mathbf{A}(n) \mathbf{w}(n))^T \mathbf{B}(n) (\mathbf{d}(n) - \mathbf{A}(n) \mathbf{w}(n)) \\ &= \mathbf{d}(n)^T \mathbf{B}(n) \mathbf{d}(n) + \mathbf{w}^T(n) \Phi(n) \mathbf{w}(n) - 2 \mathbf{w}^T(n) \varphi(n) \end{aligned} \quad (4.18)$$

where we have defined $\Phi(n)$ and $\varphi(n)$ as

$$\Phi(n) = \mathbf{A}^T(n) \mathbf{B}(n) \mathbf{A}(n) \quad (4.19)$$

$$\varphi(n) = \mathbf{A}^T(n) \mathbf{B}(n) \mathbf{d}(n) . \quad (4.20)$$

We refer to $\Phi(n)$ and $\varphi(n)$ as the correlation matrix and the cross-correlation vector, respectively, since they are scaled and weighted estimates of $\mathbf{R}_u$ and $\mathbf{r}_{ud}$.

2. Find the gradient

$$\mathbf{g}(\mathbf{w}(n)) = (\Phi(n) + \Phi^T(n)) \mathbf{w}(n) - 2 \varphi(n) = 2 \Phi(n) \mathbf{w}(n) - 2 \varphi(n) \quad (4.21)$$

3. Solve $\mathbf{g}(\mathbf{w}(n)) = \mathbf{0}$ for $\mathbf{w}(n)$

$$\begin{aligned} &\mathbf{g}(\mathbf{w}(n)) = 2 \Phi(n) \mathbf{w}(n) - 2 \varphi(n) \\ \Downarrow & \\ &\boxed{\Phi(n) \mathbf{w}(n) = \varphi(n)} \quad (\text{Weighted Normal Equations}) \end{aligned} \quad (4.22)$$

$$\Downarrow \quad \mathbf{w}(n) = \Phi^{-1}(n) \varphi(n) \quad (\text{If } \Phi(n) \text{ is invertible}) \quad (4.23)$$

4. Find the Hessian

$$\mathbf{H}(\mathbf{w}) = 2 \Phi(n) \quad (4.24)$$

which is p.d. for all $\mathbf{w}(n)$ if $\mathbf{A}(n)$ has full rank and $\beta(n, i) > 0$ for all $n \geq i > 0$.

5. This implies that

- $J_\beta(\mathbf{w}(n))$ is a convex function, and
- $\mathbf{w}_o(n) = \Phi^{-1}(n) \varphi(n)$ is the global minimiser.

The solution $\mathbf{w}_o(n)$ is often referred to as the weighted least-squares solution.

Estimation of the (Cross-)Correlation

Comparing the weighted normal equations with the Wiener-Hopf equations, we see that

$$\hat{\mathbf{R}}_u(n) = c(n, \beta) \Phi(n) = c(n, \beta) \mathbf{A}^T(n) \mathbf{B}(n) \mathbf{A}(n) = c(n, \beta) \sum_{i=1}^n \beta(n, i) \mathbf{u}(i) \mathbf{u}^T(i) \quad (4.25)$$

$$\hat{\mathbf{r}}_{ud}(n) = c(n, \beta) \boldsymbol{\varphi}(n) = c(n, \beta) \mathbf{A}^T(n) \mathbf{B}(n) \mathbf{d}(n) = c(n, \beta) \sum_{i=1}^n \beta(n, i) \mathbf{u}(i) d(i) \quad (4.26)$$

are the estimates of the correlation matrix and the cross-correlation vector, respectively. The constant $c(n, \beta)$ depends on n and the weighting function $\beta(n, i)$, and it can be selected such that $\hat{\mathbf{R}}_u(n)$ and $\hat{\mathbf{r}}_{ud}(n)$ are unbiased estimates of $\mathbf{R}_u(n)$ and $\mathbf{r}_{ud}(n)$.

4.2.2 Weight Functions

We consider three simple weight functions.

Growing Window Weight Function

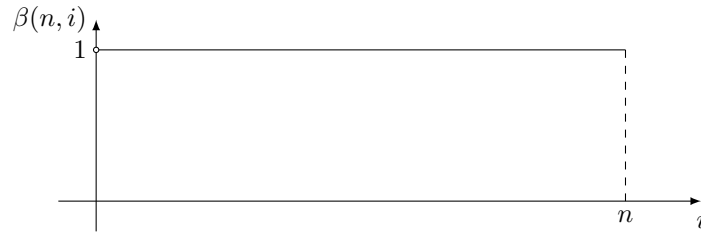


Figure 4.2: The growing window weight function.

If we select the weight function as

$$\beta(n, i) = \begin{cases} 1 & 0 < i \leq n \\ 0 & \text{otherwise} \end{cases}, \quad (4.27)$$

we obtain the growing window weight function, and it is sketched in Fig. 4.2. Selecting the growing window weight function reduces the weighted least-squares problem to the standard least-squares problem. In order to obtain unbiased estimates of $\mathbf{R}_u(n)$ and $\mathbf{r}_{ud}(n)$ in Eq. (4.25) and Eq. (4.26), respectively, we have to use

$$c(n, \beta) = \frac{1}{n}. \quad (4.28)$$

Sliding Window Weight Function

If we select the weight function as

$$\beta(n, i) = \begin{cases} 1 & n - L < i \leq n \\ 0 & \text{otherwise} \end{cases} \quad (4.29)$$

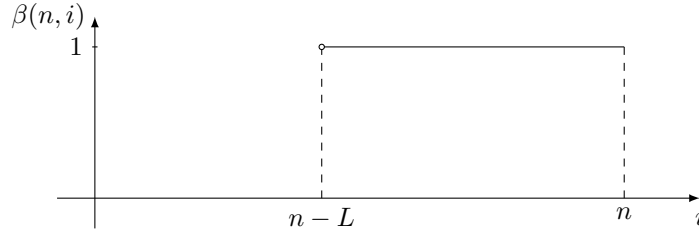


Figure 4.3: The sliding window weight function.

for $0 < L \leq n$, we obtain the sliding window weight function, and it is sketched in Fig. 4.3. If we select $L = n$, the sliding window weight function reduces to the growing window weight function. In order to obtain unbiased estimates of $\mathbf{R}_u(n)$ and $\mathbf{r}_{ud}(n)$ in Eq. (4.25) and Eq. (4.26), respectively, we have to use

$$c(n, \beta) = \frac{1}{L} . \quad (4.30)$$

Exponential Weight Function

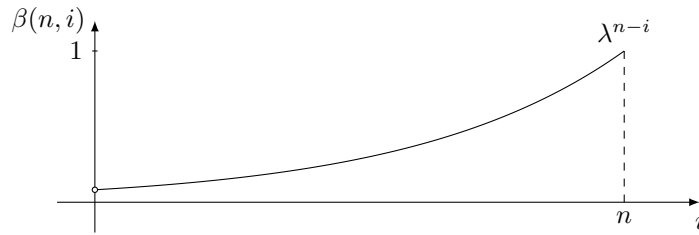


Figure 4.4: The exponential weight function.

If we select the weight function as

$$\beta(n, i) = \begin{cases} \lambda^{n-i} & 0 < i \leq n \\ 0 & \text{otherwise} \end{cases} \quad (4.31)$$

for $0 < \lambda \leq 1$, we obtain the exponential weight function, and it is sketched in Fig. 4.4. The parameter λ is called *the forgetting factor*. If we select $\lambda = 1$, the exponential weight function reduces to the growing window weight function. In order to obtain unbiased estimates of $\mathbf{R}_u(n)$ and $\mathbf{r}_{ud}(n)$ in Eq. (4.25) and Eq. (4.26), respectively, we have to use

$$c(n, \beta) = \begin{cases} \frac{1 - \lambda}{1 - \lambda^n} & 0 < \lambda < 1 \\ \frac{1}{n} & \lambda = 1 \end{cases} . \quad (4.32)$$

4.3 The Recursive Least-Squares Algorithm with an Exponential Weight Function

In an online algorithm, we have to solve the weighted normal equations

$$\Phi(n)\mathbf{w}(n) = \varphi(n) \quad (4.33)$$

for $\mathbf{w}(n)$ at every time index n . However, solving this equation directly as

$$\mathbf{w}(n) = \Phi^{-1}(n)\varphi(n) \quad (4.34)$$

yields a high computational complexity of the algorithm for the following two reasons.

1. The matrices $\mathbf{A}(n)$ and $\mathbf{B}(n)$, and the vector $\mathbf{d}(n)$ grows with n . Computing $\Phi(n)$ and $\varphi(n)$ directly would therefore be infeasible for an online algorithm, unless we use a weight function with a finite duration window.
2. For an M -tap FIR filter, it requires in the order of $\mathcal{O}(M^3)$ operations to solve the normal equations for the filter coefficient vector $\mathbf{w}(n)$.

The recursive least-squares (RLS) algorithm bypasses these two problems. Below, we consider how this is obtained for an exponential weight function, which is the most common weight function.

Recursive Computation of $\Phi(n)$ and $\varphi(n)$

We may compute the correlation matrix recursively by rewriting it as

$$\Phi(n) = \mathbf{A}^T(n)\mathbf{B}(n)\mathbf{A}(n) = \sum_{i=1}^n \lambda^{n-i} \mathbf{u}(i)\mathbf{u}^T(i) \quad (4.35)$$

$$\begin{aligned} &= \lambda^0 \mathbf{u}(n)\mathbf{u}^T(n) + \sum_{i=1}^{n-1} \lambda^{n-i} \mathbf{u}(i)\mathbf{u}^T(i) = \mathbf{u}(n)\mathbf{u}^T(n) + \lambda \sum_{i=1}^{n-1} \lambda^{n-1-i} \mathbf{u}(i)\mathbf{u}^T(i) \\ &= \mathbf{u}(n)\mathbf{u}^T(n) + \lambda \Phi(n-1) . \end{aligned} \quad (4.36)$$

Similarly, we may compute the cross-correlation vector recursively by rewriting it as

$$\varphi(n) = \mathbf{A}^T(n)\mathbf{B}(n)\mathbf{d}(n) = \sum_{i=1}^n \lambda^{n-i} \mathbf{u}(i)d(i) \quad (4.37)$$

$$\begin{aligned} &= \lambda^0 \mathbf{u}(n)d(n) + \sum_{i=1}^{n-1} \lambda^{n-i} \mathbf{u}(i)d(i) = \mathbf{u}(n)d(n) + \lambda \sum_{i=1}^{n-1} \lambda^{n-1-i} \mathbf{u}(i)d(i) \\ &= \mathbf{u}(n)d(n) + \lambda \varphi(n-1) . \end{aligned} \quad (4.38)$$

These recursive formulations of the correlation matrix and the cross-correlation vector clearly reduce the computational complexity and are suitable to use in an online algorithm.

Inversion of $\Phi(n)$

For the inversion of the correlation matrix, we use the matrix inversion lemma stated in Sec. 4.1.1. Comparing the recursive formulation of the correlation matrix in Eq. (4.36) with the left side of Eq. (4.5), we obtain

$$\mathbf{X} = \lambda \Phi(n-1) \quad (4.39)$$

$$\mathbf{U} = \mathbf{u}(n) \quad (4.40)$$

$$\mathbf{V} = \mathbf{u}^T(n) \quad (4.41)$$

$$\mathbf{Y} = 1 \quad (4.42)$$

Thus, invoking the matrix inversion lemma, we have that

$$\Phi^{-1}(n) = \lambda^{-1} \Phi^{-1}(n-1) - \lambda^{-2} \frac{\Phi^{-1}(n-1) \mathbf{u}(n) \mathbf{u}^T(n) \Phi^{-1}(n-1)}{1 + \lambda^{-1} \mathbf{u}^T(n) \Phi^{-1}(n-1) \mathbf{u}(n)} \quad (4.43)$$

Note that the computational complexity of the right side of the equation is much lower than that of the left side of the equation when $\Phi^{-1}(n-1)$ is known. In order to simplify the notation, we define

$$\mathbf{P}(n) = \Phi^{-1}(n) \quad (\text{inverse correlation matrix}) \quad (4.44)$$

$$\mathbf{k}(n) = \frac{\mathbf{P}(n-1) \mathbf{u}(n)}{\lambda + \mathbf{u}^T(n) \mathbf{P}(n-1) \mathbf{u}(n)} \quad (\text{gain vector}) \quad (4.45)$$

which leads to that we may write Eq. (4.43) as

$$\mathbf{P}(n) = \lambda^{-1} [\mathbf{P}(n-1) - \mathbf{k}(n) \mathbf{u}^T(n) \mathbf{P}(n-1)] \quad (4.46)$$

By rearranging the expression for the gain vector, we obtain

$$\begin{aligned} \mathbf{k}(n) &= \lambda^{-1} [\mathbf{P}(n-1) - \mathbf{k}(n) \mathbf{u}^T(n) \mathbf{P}(n-1)] \mathbf{u}(n) \\ &= \mathbf{P}(n) \mathbf{u}(n) \end{aligned} \quad (4.47)$$

Recursive computation of $\mathbf{w}(n)$

We can now develop the recursive update equation of the filter coefficient vector. We have that

$$\mathbf{w}(n) = \mathbf{P}(n) \boldsymbol{\varphi}(n) \quad (4.48)$$

$$\begin{aligned} &= \mathbf{P}(n) [\mathbf{u}(n) d(n) + \lambda \boldsymbol{\varphi}(n-1)] = \lambda \mathbf{P}(n) \mathbf{P}^{-1}(n-1) \mathbf{w}(n-1) + \mathbf{k}(n) d(n) \\ &= [\mathbf{P}(n-1) - \mathbf{k}(n) \mathbf{u}^T(n) \mathbf{P}(n-1)] \mathbf{P}^{-1}(n-1) \mathbf{w}(n-1) + \mathbf{k}(n) d(n) \\ &= \mathbf{w}(n-1) - \mathbf{k}(n) \mathbf{u}^T(n) \mathbf{w}(n-1) + \mathbf{k}(n) d(n) \\ &= \mathbf{w}(n-1) + \mathbf{k}(n) \xi(n) \end{aligned} \quad (4.49)$$

where we have defined the a priori error as

$$\xi(n) = d(n) - \mathbf{u}^T(n) \mathbf{w}(n-1) \quad (4.50)$$

The RLS Algorithm

The RLS algorithm may now be formulated as the following set of equations

$$\boldsymbol{\pi}(n) = \mathbf{P}(n-1)\mathbf{u}(n) \quad (4.51)$$

$$\mathbf{k}(n) = \frac{\boldsymbol{\pi}(n)}{\lambda + \mathbf{u}^T(n)\boldsymbol{\pi}(n)} \quad (4.52)$$

$$\xi(n) = d(n) - \mathbf{u}^T(n)\mathbf{w}(n-1) \quad (4.53)$$

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n)\xi(n) \quad (4.54)$$

$$\mathbf{P}(n) = \lambda^{-1} [\mathbf{P}(n-1) - \mathbf{k}(n)\boldsymbol{\pi}^T(n)] \quad (4.55)$$

For an M -tap FIR filter, it requires in the order of $\mathcal{O}(M^2)$ operations to run one iteration of the RLS algorithm.

Initialisation

In order to start the RLS algorithm, we need to select values for the initial inverse correlation matrix $\mathbf{P}(0)$, the initial filter coefficient vector $\mathbf{w}(0)$, and the input samples $u(n)$ for $n = -M + 1, -M + 2, \dots, 1$. Typically, we assume that

$$\mathbf{P}(0) = \delta^{-1}\mathbf{I} \quad (4.56)$$

$$\mathbf{w}(0) = \mathbf{0} \quad (4.57)$$

$$u(n) = 0, \quad \text{for } -M + 1 < n < 1. \quad (4.58)$$

The first assumption implies that we assume that $\mathbf{u}(n)$ for $n < 1$ is a white random process with covariance matrix $\delta\mathbf{I}$. The value of δ should reflect the SNR of the input data with δ being small for a high SNR and δ being large for a low SNR [8, pp. 444–446]. This assumption introduces bias into the correlation matrix $\boldsymbol{\Phi}(n)$. However, this bias decreases to zero for an increasing n . An alternative initialisation, which does not introduce bias, is to estimate the correlation matrix and the cross-correlation vector as [10, pp. 545–546]

$$\mathbf{P}(0) = \left[\sum_{i=-M+1}^0 \lambda^{-i} \mathbf{u}(i)\mathbf{u}^T(i) \right]^{-1} \quad (4.59)$$

$$\boldsymbol{\varphi}(0) = \sum_{i=-M+1}^0 \lambda^{-i} \mathbf{u}(i)d(i) \quad (4.60)$$

prior to starting the RLS algorithm at time $n = 1$. The initial value of the filter coefficient vector can be set to $\mathbf{w}(0) = \mathbf{P}(0)\boldsymbol{\varphi}(0)$. Note, that this approach requires that we now the input signal from time $n = -2M + 2$ and the desired signal from time $n = -M + 2$.

4.3.1 Selection of the Forgetting Factor

At time n , the memory of the sliding window RLS algorithm is the L newest samples indexed by $n - L + 1, \dots, n$. For the exponentially weighted RLS algorithm, the memory is controlled by the forgetting factor λ . Whereas the interpretation of L is simple, the corresponding interpretation of λ is not that intuitive when we have to investigate the memory of the exponentially weighted RLS algorithm. That is, we would like to interpret the forgetting factor as a sliding window

length. We call this window length for the effective window length and denote it by L_{eff} . A simple way of connecting L_{eff} and λ is by requiring that

$$\lim_{n \rightarrow \infty} \sum_{i=1}^n \lambda^{n-i} = \lim_{n \rightarrow \infty} \sum_{i=n-L_{\text{eff}}+1}^n 1. \quad (4.61)$$

That is, when the RLS algorithm has reached steady-state, the area under the sliding window curve should equal the area under the exponential window curve. This leads to

$$\lim_{n \rightarrow \infty} \sum_{i=n-L_{\text{eff}}+1}^n 1 = L_{\text{eff}} \quad (4.62)$$

and

$$\lim_{n \rightarrow \infty} \sum_{i=1}^n \lambda^{n-i} = \lim_{n \rightarrow \infty} \sum_{k=0}^{n-1} \lambda^k = \lim_{n \rightarrow \infty} \sum_{k=0}^{n-1} \lambda^k = \lim_{n \rightarrow \infty} \frac{1 - \lambda^n}{1 - \lambda} = \frac{1}{1 - \lambda} \quad (4.63)$$

where the second last equality follows for $\lambda \neq 1$ from the geometric series, and the last equality follows if $0 < \lambda < 1$. Thus, we have that

$$L_{\text{eff}} = \frac{1}{1 - \lambda}. \quad (4.64)$$

4.3.2 Transient Analysis

The RLS algorithm is stable in the mean and the mean-square if $0 < \lambda \leq 1$. It may also be shown that the rate of the convergence of the RLS algorithm is typically an order of magnitude faster than the rate of the convergence of the LMS algorithm. Moreover, the rate of the convergence of the RLS algorithm is invariant to the condition number of the correlation matrix $\mathbf{R}_u$ of the input signal [8, p. 463, ch. 14].

4.3.3 Steady-State Analysis

It can be shown that [7, p. 510]

$$\begin{aligned} \text{EMSE:} \quad J_{\text{ex}} &= J_2(\mathbf{w}(\infty)) - J_{\min} \\ &\approx J_{\min} \frac{(1 - \lambda)M}{1 + \lambda - (1 - \lambda)M} \end{aligned} \quad (4.65)$$

$$\text{Misadjustment:} \quad \mathcal{M} = \frac{J_{\text{ex}}}{J_{\min}} \approx \frac{(1 - \lambda)M}{1 + \lambda - (1 - \lambda)M} \quad (4.66)$$

$$\text{MSD:} \quad E[\|\Delta \mathbf{w}(\infty)\|^2] \approx J_{\text{ex}} \sum_{m=1}^M \frac{1}{\lambda_m} \quad (4.67)$$

where λ_m is the m 'th eigenvalue of the correlation matrix $\mathbf{R}_u$ not to be confused with the forgetting factor λ . The approximations hold under certain conditions which may be found in [7, pp. 508–510].

4.3.4 Computational Cost

Table 4.1 shows the computational cost of the RLS algorithm in terms of the number of multiplications, additions or subtractions, and divisions. From the table, we see that the total number of flops is $5M^2 + 6M$. Thus, the RLS algorithm has a complexity of $\mathcal{O}(M^2)$. Note, that there exist faster ways of implementing the RLS algorithm [7, pp. 247]. Some of them even achieve linear complexity [11].

Term	$\times$	$+$ or $-$	$/$
$\boldsymbol{\pi}(n) = \mathbf{P}(n-1)\mathbf{u}(n)$	M^2	$M(M-1)$	
$\mathbf{k}(n) = \boldsymbol{\pi}(n)/(\lambda + \mathbf{u}^T(n)\boldsymbol{\pi}(n))$	M	M	M
$\xi(n) = d(n) - \mathbf{u}^T(n)\mathbf{w}(n-1)$	M	M	
$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n)\xi(n)$	M	M	
$\mathbf{P}(n) = (\mathbf{P}(n-1) - \mathbf{k}(n)\boldsymbol{\pi}^T(n))/\lambda$	M^2	M^2	M^2
Total	$2M^2 + 3M$	$2M^2 + 2M$	$M^2 + M$

Table 4.1: Computational cost of the RLS algorithm.

Bibliography

- [1] D. C. Lay, *Linear algebra and its applications*, 3rd ed. Pearson/Addison-Wesley, Sep. 2005.
- [2] G. Strang, *Introduction to Linear Algebra*, 4th ed. Wellesley Cambridge Press, Feb. 2009.
- [3] A. Antoniou and W.-S. Lu, *Practical Optimization: Algorithms and Engineering Applications*. Springer, Mar. 2007.
- [4] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, Mar. 2004.
- [5] S. M. Kay, *Intuitive Probability and Random Processes using MATLAB*. Springer Science, New York, Inc., 2006.
- [6] A. Leon-Garcia, *Probability, Statistics, and Random Processes For Electrical Engineering*, 3rd ed. Prentice Hall, Jan. 2008.
- [7] A. H. Sayed, *Fundamentals of Adaptive Filtering*. Wiley-IEEE Press, Jun. 2003.
- [8] S. Haykin, *Adaptive Filter Theory*, 4th ed. Prentice Hall, Sep. 2001.
- [9] H.-C. Shin and A. H. Sayed, “Mean-square performance of a family of affine projection algorithms,” *IEEE Trans. Signal Process.*, vol. 52, no. 1, pp. 90–102, Jan. 2004.
- [10] M. H. Hayes, *Statistical Digital Signal Processing and Modeling*. Wiley, 1996.
- [11] D. T. M. Slock and T. Kailath, “Numerically stable fast transversal filters for recursive least squares adaptive filtering,” *IEEE Trans. Signal Process.*, vol. 39, no. 1, pp. 92–114, Jan. 1991.
- [12] K. B. Petersen and M. S. Pedersen, “The matrix cookbook,” Online, oct 2008, version 20081110. [Online]. Available: <http://www2.imm.dtu.dk/pubdb/p.php?3274>

Appendix A

Summary

Fig. A.1 shows an overview over the various important concepts, equations and adaptive filters covered in these lecture notes. All relationships indicated by the arrows have been covered with the exception of the arrow from the steepest descent block to the NLMS block. The interested reader may establish this relationship by solving Problem 1 of Chapter 6 in [8].

Table A.1 shows a summary of the adaptive filters covered in these lecture notes. Most the expressions for the mean-square stability, the excess mean-square error (EMSE), the misadjustment, and the mean-square deviation (MSD) are approximations, and they can therefore only be used as a rule of thumb. For more accurate expressions see the description of the adaptive filters and the references therein.

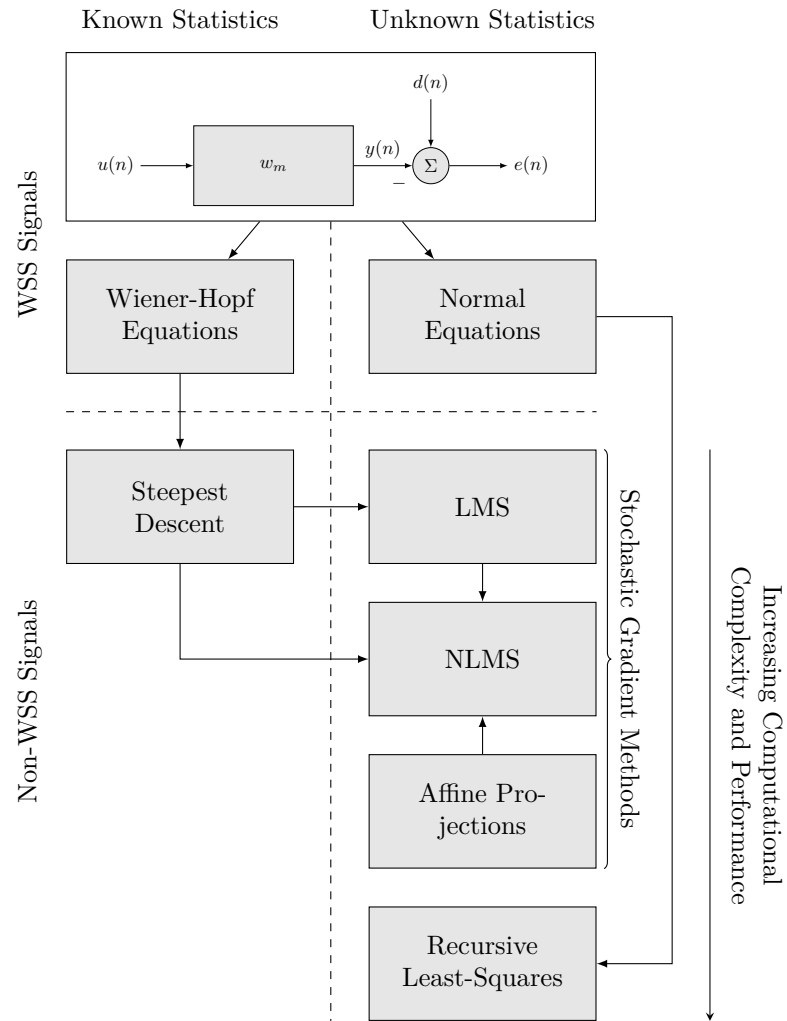


Figure A.1: Overview over the various important concepts, equations and adaptive filters covered in these lecture notes.

Name	Algorithm	Cost	Mean-Square Stability	EMSE	Misadjustment	MSD
SD	$\mathbf{g}(\mathbf{w}(n)) = 2\mathbf{R}_u\mathbf{w}(n) - 2\mathbf{r}_{ud}(n)$	$\mathcal{O}(M)$	$0 < \mu < \frac{2}{\lambda_{\max}}$	0	0	0
	$\mathbf{w}(n+1) = \mathbf{w}(n) - \frac{\mu}{2}\mathbf{g}(\mathbf{w}(n))$					
LMS	$e(n) = d(n) - \mathbf{u}^T(n)\mathbf{w}(n)$	$\mathcal{O}(M)$	$0 < \mu < \frac{2}{\text{tr}(\mathbf{R}_u)}$	$\frac{\mu}{2}J_{\min}\text{tr}(\mathbf{R}_u)$	$\frac{\mu}{2}\text{tr}(\mathbf{R}_u)$	$\frac{\mu}{2}J_{\min}M$
	$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu\mathbf{u}(n)e(n)$					
NLMS	$e(n) = d(n) - \mathbf{u}^T(n)\mathbf{w}(n)$	$\mathcal{O}(M)$	$0 < \beta < 2$	$\frac{\beta}{2}J_{\min}$	$\frac{\beta}{2}$	$\frac{\beta J_{\min}}{2\text{tr}(\mathbf{R}_u)}$
	$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\beta}{\epsilon + \ \mathbf{u}(n)\ ^2}\mathbf{u}(n)e(n)$					
APA	$\mathbf{e}(n) = \mathbf{d}(n) - \mathbf{U}^T(n)\mathbf{w}(n)$	$\mathcal{O}(MK^2)$	$0 < \beta < 2$	$\frac{\beta}{2}J_{\min}K$	$\frac{\beta}{2}K$	no simple expression
	$\mathbf{w}(n+1) = \mathbf{w}(n) + \beta\mathbf{U}(n)[\epsilon\mathbf{I} + \mathbf{U}^T(n)\mathbf{U}(n)]^{-1}\mathbf{e}(n)$					
RLS	$\boldsymbol{\pi}(n) = \mathbf{P}(n-1)\mathbf{u}(n)$	$\mathcal{O}(M^2)$	$0 < \lambda \leq 1$	$\frac{J_{\min}\frac{1-\lambda}{1+\lambda}M}{1 - \frac{1-\lambda}{1+\lambda}M}$	$\frac{\frac{1-\lambda}{1+\lambda}M}{1 - \frac{1-\lambda}{1+\lambda}M}$	$\frac{J_{\min}\frac{1-\lambda}{1+\lambda}M}{1 - \frac{1-\lambda}{1+\lambda}M} \sum_{m=1}^M \frac{1}{\lambda_m}$
	$\mathbf{k}(n) = \frac{\boldsymbol{\pi}(n)}{\lambda + \mathbf{u}^T(n)\boldsymbol{\pi}(n)}$					
	$\boldsymbol{\xi}(n) = \mathbf{d}(n) - \mathbf{u}^T(n)\mathbf{w}(n-1)$					
	$\mathbf{w}(n+1) = \mathbf{w}(n-1) + \mathbf{k}(n)\boldsymbol{\xi}(n)$					
	$\mathbf{P}(n) = \lambda^{-1}[\mathbf{P}(n-1) - \mathbf{k}(n)\boldsymbol{\pi}^T(n)]$					

Table A.1: Overview over the basic adaptive filters and their properties. Most of the expressions for the mean-square stability, the excess mean-square error (EMSE), the misadjustment, and the mean-square deviation (MSD) are approximations.

Appendix B

Transient and Steady-State Analysis of the LMS Adaptive Filter

In this appendix, we give a more detailed transient and steady-state analysis of the LMS adaptive filter than found in Lecture 2. Specifically, we derive the condition for mean-square convergence and the expressions for the learning curve, the excess mean-square error (EMSE), the mean-square deviation (MSD), and the misadjustment. We do this for two reasons. Firstly, it is instructive to see how the analysis is carried out, and, secondly, it highlights how difficult the analysis is - even of the simple LMS adaptive filter.

In the derivation below, we use the analysis model previously discussed in Sec. 3.2.1. We also need the following important result.

B.1 A Special Fourth Order Moment of a Gaussian Random Vector

Let $\mathbf{u}$ be a real-valued $M \times 1$ dimensional Gaussian random vector with zero-mean and correlation matrix $\mathbf{R}_u$. Moreover, let $\mathbf{W}$ be an $M \times M$ symmetric matrix. We then have that [7, p. 44]

$$E[\mathbf{u}\mathbf{u}^T \mathbf{W} \mathbf{u}\mathbf{u}^T] = \mathbf{R}_u \text{tr}(\mathbf{W} \mathbf{R}_u) + 2\mathbf{R}_u \mathbf{W} \mathbf{R}_u . \quad (\text{B.1})$$

If $\mathbf{R}_u = \mathbf{X} \mathbf{\Lambda} \mathbf{X}^T$ is the eigenvalue decomposition of the correlation matrix and if we define $\mathbf{F} = \mathbf{X}^T \mathbf{W} \mathbf{X}$, we may write

$$E[\mathbf{u}\mathbf{u}^T \mathbf{W} \mathbf{u}\mathbf{u}^T] = \mathbf{X} [\mathbf{\Lambda} \text{tr}(\mathbf{F} \mathbf{\Lambda}) + 2\mathbf{\Lambda} \mathbf{F} \mathbf{\Lambda}] \mathbf{X}^T . \quad (\text{B.2})$$

With this result in mind, we can now evaluate the following expectation

$$E[(\mathbf{I} - \mu \mathbf{u}\mathbf{u}^T) \mathbf{W} (\mathbf{I} - \mu \mathbf{u}\mathbf{u}^T)] = \mathbf{W} + \mu^2 E[\mathbf{u}\mathbf{u}^T \mathbf{W} \mathbf{u}\mathbf{u}^T] - \mu \mathbf{R}_u \mathbf{W} - \mu \mathbf{W} \mathbf{R}_u \quad (\text{B.3})$$

$$= \mathbf{W} + \mu^2 \mathbf{R}_u \text{tr}(\mathbf{W} \mathbf{R}_u) + 2\mu^2 \mathbf{R}_u \mathbf{W} \mathbf{R}_u - \mu \mathbf{R}_u \mathbf{W} - \mu \mathbf{W} \mathbf{R}_u \quad (\text{B.4})$$

$$= \mathbf{X} [\mathbf{F} - \mu(\mathbf{\Lambda} \mathbf{F} + \mathbf{F} \mathbf{\Lambda}) + \mu^2(\mathbf{\Lambda} \text{tr}(\mathbf{F} \mathbf{\Lambda}) + 2\mathbf{\Lambda} \mathbf{F} \mathbf{\Lambda})] \mathbf{X}^T . \quad (\text{B.5})$$

If $\mathbf{F}$ is diagonal, then $\mathbf{F}\mathbf{\Lambda} = \mathbf{\Lambda}\mathbf{F}$ and we have that

$$E[(\mathbf{I} - \mu\mathbf{u}\mathbf{u}^T)\mathbf{W}(\mathbf{I} - \mu\mathbf{u}\mathbf{u}^T)] = \mathbf{X}[\mathbf{F} - 2\mu\mathbf{\Lambda}\mathbf{F} + \mu^2(\mathbf{\Lambda}\text{tr}(\mathbf{F}\mathbf{\Lambda}) + 2\mathbf{\Lambda}^2\mathbf{F})]\mathbf{X}^T. \quad (\text{B.6})$$

This result is very important for the derivation below.

B.2 The Analysis Model

The LMS algorithm is given by

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu\mathbf{u}(n)e(n) \quad (\text{B.7})$$

where $\mathbf{w}(n)$, $\mathbf{u}(n)$, and $e(n)$ are the filter vector, the input vector and the error, respectively, at time index n . The positive scalar μ is the step-size, and the error is given by

$$e(n) = d(n) - \mathbf{u}^T(n)\mathbf{w}(n) \quad (\text{B.8})$$

where the desired signal $d(n)$ in the analysis model (see also Sec. 3.2.1) is given by

$$d(n) = v(n) + \mathbf{u}^T(n)\mathbf{w}_o. \quad (\text{B.9})$$

We assume that $v(n)$ is white Gaussian noise with variance σ_v^2 and uncorrelated with the input signal $u(n)$. Finally, we assume that $\mathbf{u}(n)$ is a white Gaussian process with correlation matrix $\mathbf{R}_u$. This implies that $\mathbf{u}(n)$ and $\mathbf{w}(n)$ are uncorrelated.

The difference between the optimal filter vector $\mathbf{w}_o$ and the filter vector $\mathbf{w}(n)$ is called the weight error, and it is given by

$$\Delta\mathbf{w}(n) = \mathbf{w}_o - \mathbf{w}(n) = \mathbf{w}_o - \mathbf{w}(n-1) - \mu\mathbf{u}(n-1)e(n-1) \quad (\text{B.10})$$

$$= \Delta\mathbf{w}(n-1) - \mu\mathbf{u}(n-1)[v(n) + \mathbf{u}^T(n-1)\mathbf{w}_o - \mathbf{u}^T(n-1)\mathbf{w}(n-1)] \quad (\text{B.11})$$

$$= [\mathbf{I} - \mu\mathbf{u}(n-1)\mathbf{u}^T(n-1)]\Delta\mathbf{w}(n-1) - \mu\mathbf{u}(n-1)v(n). \quad (\text{B.12})$$

In terms of the weight error, the error is given by

$$e(n) = d(n) - \mathbf{u}^T(n)\mathbf{w}(n) = v(n) + \mathbf{u}^T(n)\Delta\mathbf{w}(n), \quad (\text{B.13})$$

and the minimum mean-squared error (MMSE) is achieved at the optimum where $\Delta\mathbf{w}(n) = \mathbf{0}$ and thus given by

$$J_{\min} = J_1(\mathbf{w}_o) = E[v^2(n)] = \sigma_v^2. \quad (\text{B.14})$$

B.3 Transient Analysis

In the transient analysis, we first consider how we should select the step-size μ in order to ensure the mean-square stability of the LMS algorithm. Second, we derive an expression for the learning curve.

B.3.1 Mean-Square Convergence

As already discussed in Lecture 2, we say that the LMS algorithm converges in the mean-square if

$$\lim_{n \rightarrow \infty} E[\|\Delta\mathbf{w}(n)\|^2] = c < \infty. \quad (\text{B.15})$$

In this section, we show that the LMS algorithm converges in the mean square if the step-size μ is selected such that it satisfies

$$f(\mu) = \frac{\mu}{2} \sum_{m=1}^M \frac{\lambda_m}{1 - \mu\lambda_m} < 1 \quad (\text{B.16})$$

where λ_m is the m 'th eigenvalue of the correlation matrix $\mathbf{R}_u$.

In order to do this, we investigate $E[\|\Delta\mathbf{w}(n)\|^2]$ as n grows. For convenience of notation, it turns out to be easier to work with $E[\|\mathbf{c}(n)\|^2]$ where $\mathbf{c}(n) = \mathbf{X}^T \Delta\mathbf{w}(n)$ and $\mathbf{X}$ contains the M eigenvectors of $\mathbf{R}_u$. Since $\mathbf{X}$ is orthogonal, it follows that

$$E[\|\mathbf{c}(n)\|^2] = E[\|\mathbf{X}^T \Delta\mathbf{w}(n)\|^2] = E[\|\Delta\mathbf{w}(n)\|^2] \quad (\text{B.17})$$

Thus, $E[\|\mathbf{c}(n)\|^2]$ converges in the same way as $E[\|\Delta\mathbf{w}(n)\|^2]$ does. In the derivation below, we express $E[\|\mathbf{c}(n)\|^2]$ as a function of $\mathbf{c}(0)$, μ , $\mathbf{\Lambda}$, and $J_{\min}$. We do this in a recursive manner starting by expressing $E[\|\mathbf{c}(n)\|^2]$ as a function of $\mathbf{c}(n-1)$, μ , $\mathbf{\Lambda}$, and $J_{\min}$. Once we have obtained an expression for this recursion, we express $E[\|\mathbf{c}(n)\|^2]$ as a function of $\mathbf{c}(n-2)$, μ , $\mathbf{\Lambda}$, and $J_{\min}$ in the next recursion and so on.

The First Recursion

Multiplying Eq. (B.12) from the left with $\mathbf{X}^T$ and inserting it into $E[\|\mathbf{c}(n)\|^2]$ yields

$$E[\|\mathbf{c}(n)\|^2] = E[\|\mathbf{X}^T [\mathbf{I} - \mu\mathbf{u}(n-1)\mathbf{u}^T(n-1)]\mathbf{X}\mathbf{c}(n-1) - \mu\mathbf{X}^T\mathbf{u}(n-1)v(n)\|^2] \quad (\text{B.18})$$

$$\begin{aligned} &= E[\mathbf{c}^T(n-1)\mathbf{X}^T[\mathbf{I} - \mu\mathbf{u}(n-1)\mathbf{u}^T(n-1)]^2\mathbf{X}\mathbf{c}(n-1)] \\ &\quad + \mu^2 E[v^2(n)\mathbf{u}^T(n-1)\mathbf{u}(n-1)] \end{aligned} \quad (\text{B.19})$$

where the last equality follows from the assumption that $v(n)$ and $\mathbf{u}(n)$ are uncorrelated. The expectation of the last term is given by

$$E[v^2(n)\mathbf{u}^T(n-1)\mathbf{u}(n-1)] = E[v^2(n)]E[\mathbf{u}^T(n-1)\mathbf{u}(n-1)] \quad (\text{B.20})$$

$$= J_{\min} E[\text{tr}(\mathbf{u}^T(n-1)\mathbf{u}(n-1))] \quad (\text{B.21})$$

$$= J_{\min} E[\text{tr}(\mathbf{u}(n-1)\mathbf{u}^T(n-1))] \quad (\text{B.22})$$

$$= J_{\min} \text{tr}(E[\mathbf{u}(n-1)\mathbf{u}^T(n-1)]) \quad (\text{B.23})$$

$$= J_{\min} \text{tr}(\mathbf{R}_u) \quad (\text{B.24})$$

$$= J_{\min} \text{tr}(\mathbf{\Lambda}) . \quad (\text{B.25})$$

The second equality follows since the trace of a scalar equals the value of the scalar. The third equality follows from a property of the trace operator given by

$$\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA}) \quad (\text{B.26})$$

for any pair of matrices $\mathbf{A}$ and $\mathbf{B}$ with $\mathbf{A}$ and $\mathbf{B}^T$ being of the same dimension. The fourth equality follows since both the expectation and the trace operators are linear operators whose order can therefore be interchanged. Finally, the last equality follows since

$$\text{tr}(\mathbf{R}_u) = \text{tr}(\mathbf{X}\mathbf{\Lambda}\mathbf{X}^T) = \text{tr}(\mathbf{\Lambda}\mathbf{X}^T\mathbf{X}) = \text{tr}(\mathbf{\Lambda}) . \quad (\text{B.27})$$

This was previously stated in the beginning of Lecture 2.

In order to evaluate the expectation of the first term in Eq. (B.19), we use the law of iterated expectations given by¹

$$E[X] = E[E[X|Y]] , \quad (\text{B.28})$$

and the assumption that $\mathbf{u}(n)$ and $\mathbf{w}(n)$ are uncorrelated. From this, we obtain that

$$\begin{aligned} E[\mathbf{c}^T(n-1)\mathbf{X}^T[\mathbf{I} - \mu\mathbf{u}(n-1)\mathbf{u}^T(n-1)]^2\mathbf{X}\mathbf{c}(n-1)] = \\ E[\mathbf{c}^T(n-1)\mathbf{X}^T E[(\mathbf{I} - \mu\mathbf{u}(n-1)\mathbf{u}^T(n-1))^2]\mathbf{X}\mathbf{c}(n-1)] . \end{aligned} \quad (\text{B.29})$$

The inner expectation is on the same form as Eq. (B.6) with $\mathbf{W} = \mathbf{F} = \mathbf{I}$. Thus, we have that

$$\mathbf{S}(1) = \mathbf{X}^T E[(\mathbf{I} - \mu\mathbf{u}(n-1)\mathbf{u}^T(n-1))^2]\mathbf{X} \quad (\text{B.30})$$

$$= \mathbf{I} - 2\mu\mathbf{\Lambda} + \mu^2[\mathbf{\Lambda}\text{tr}(\mathbf{\Lambda}) + 2\mathbf{\Lambda}^2] . \quad (\text{B.31})$$

which is a diagonal matrix. Thus, we can write Eq. (B.19) as

$$E[\|\mathbf{c}(n)\|^2] = E[\mathbf{c}^T(n-1)\mathbf{S}(1)\mathbf{c}(n-1)] + \mu^2 J_{\min}\text{tr}(\mathbf{\Lambda}) . \quad (\text{B.32})$$

The Second Recursion

Since the second term of Eq. (B.32) does not depend on $\mathbf{c}(n)$, we consider the first term of Eq. (B.32). Multiplying Eq. (B.12) for time index $n-1$ from the left with $\mathbf{X}^T$ and inserting it into the first term yields

$$\begin{aligned} E[\mathbf{c}^T(n-1)\mathbf{S}(1)\mathbf{c}(n-1)] = \\ E[\mathbf{c}^T(n-2)\mathbf{X}^T[\mathbf{I} - \mu\mathbf{u}(n-2)\mathbf{u}^T(n-2)]\mathbf{X}\mathbf{S}(1)\mathbf{X}^T[\mathbf{I} - \mu\mathbf{u}(n-2)\mathbf{u}^T(n-2)]\mathbf{X}\mathbf{c}(n-2)] \\ + \mu^2 E[v^2(n)\mathbf{u}^T(n-2)\mathbf{X}\mathbf{S}(1)\mathbf{X}^T\mathbf{u}(n-2)] . \end{aligned} \quad (\text{B.33})$$

The expectation of the last term is given by

$$\begin{aligned} E[v^2(n)\mathbf{u}^T(n-2)\mathbf{X}\mathbf{S}(1)\mathbf{X}^T\mathbf{u}(n-2)] &= E[v^2(n)]E[\mathbf{u}^T(n-2)\mathbf{X}\mathbf{S}(1)\mathbf{X}^T\mathbf{u}(n-2)] \\ &= J_{\min}E[\text{tr}(\mathbf{u}^T(n-2)\mathbf{X}\mathbf{S}(1)\mathbf{X}^T\mathbf{u}(n-2))] \\ &= J_{\min}\text{tr}(\mathbf{X}\mathbf{S}(1)\mathbf{X}^T E[\mathbf{u}(n-2)\mathbf{u}^T(n-2)]) \\ &= J_{\min}\text{tr}(\mathbf{S}(1)\mathbf{\Lambda}) . \end{aligned} \quad (\text{B.34})$$

The equalities in the derivation above follows from the same arguments as in the derivation for the equivalent expression in the first recursion. Using the same arguments as in the first recursion, the first term of Eq. (B.33) may be written as an outer and an inner expectation. The inner expectation can be evaluated using Eq. (B.6) with $\mathbf{W} = \mathbf{X}\mathbf{S}(1)\mathbf{X}^T$ or $\mathbf{F} = \mathbf{S}(1)$. Thus, we have that $\mathbf{S}(2)$ is given by

$$\begin{aligned} \mathbf{S}(2) &= \mathbf{X}^T E\left[\left[\mathbf{I} - \mu\mathbf{u}(n-2)\mathbf{u}^T(n-2)\right]\mathbf{X}\mathbf{S}(1)\mathbf{X}^T\left[\mathbf{I} - \mu\mathbf{u}(n-2)\mathbf{u}^T(n-2)\right]\right]\mathbf{X} \\ &= \mathbf{S}(1) - 2\mu\mathbf{\Lambda}\mathbf{S}(1) + \mu^2[\mathbf{\Lambda}\text{tr}(\mathbf{S}(1)\mathbf{\Lambda}) + 2\mathbf{\Lambda}\mathbf{S}(1)\mathbf{\Lambda}] \end{aligned} \quad (\text{B.35})$$

¹It follows since

$$\begin{aligned} E[E[X|Y]] &= \int E[X|Y]f_Y(y)dy = \int \left[\int xf_{X|Y}(x|y)dx \right] f_Y(y)dy = \int \int xf_{X,Y}(x,y)dxdy \\ &= \int x \left[\int f_{X,Y}(x,y)dy \right] dx = \int xf_X(x)dx = E[X] . \end{aligned}$$

which is diagonal since $\mathbf{S}(1)$ is diagonal. Thus, in total we have that

$$E[\|\mathbf{c}(n)\|^2] = E[\mathbf{c}^T(n-1)\mathbf{S}(1)\mathbf{c}(n-1)] + \mu^2 J_{\min} \text{tr}(\mathbf{\Lambda}) \quad (\text{B.36})$$

$$= E[\mathbf{c}^T(n-2)\mathbf{S}(2)\mathbf{c}(n-2)] + \mu^2 J_{\min} \text{tr}(\mathbf{S}(1)\mathbf{\Lambda}) + \mu^2 J_{\min} \text{tr}(\mathbf{\Lambda}) \quad (\text{B.37})$$

$$= E[\mathbf{c}^T(n-2)\mathbf{S}(2)\mathbf{c}(n-2)] + \mu^2 J_{\min} \text{tr}([\mathbf{S}(1) + \mathbf{I}]\mathbf{\Lambda}) . \quad (\text{B.38})$$

The Third Recursion

Since the second term of Eq. (B.38) does not depend on $\mathbf{c}(n)$, we consider the first term of Eq. (B.38). This has the same form as the first term of Eq. (B.32) so the derivation of the third recursion is the same as for the second recursion, except that all the time indices should be decreased by one. We therefore obtain that

$$E[\|\mathbf{c}(n)\|^2] = E[\mathbf{c}^T(n-1)\mathbf{S}(1)\mathbf{c}(n-1)] + \mu^2 J_{\min} \text{tr}(\mathbf{\Lambda}) \quad (\text{B.39})$$

$$= E[\mathbf{c}^T(n-2)\mathbf{S}(2)\mathbf{c}(n-2)] + \mu^2 J_{\min} \text{tr}([\mathbf{S}(1) + \mathbf{I}]\mathbf{\Lambda}) \quad (\text{B.40})$$

$$= E[\mathbf{c}^T(n-3)\mathbf{S}(3)\mathbf{c}(n-3)] + \mu^2 J_{\min} \text{tr}([\mathbf{S}(2) + \mathbf{S}(1) + \mathbf{I}]\mathbf{\Lambda}) . \quad (\text{B.41})$$

The n 'th Recursion

From the first, second, and third recursion, it is not hard to see a pattern for the recursions. Therefore, for the n 'th recursion, we have that

$$E[\|\mathbf{c}(n)\|^2] = \mathbf{c}^T(0)\mathbf{S}(n)\mathbf{c}(0) + \mu^2 J_{\min} \sum_{i=0}^{n-1} \text{tr}(\mathbf{S}(i)\mathbf{\Lambda}) \quad (\text{B.42})$$

where

$$\mathbf{S}(n) = \mathbf{S}(n-1) - 2\mu\mathbf{\Lambda}\mathbf{S}(n-1) + \mu^2[\mathbf{\Lambda}\text{tr}(\mathbf{S}(n-1)\mathbf{\Lambda}) + 2\mathbf{\Lambda}\mathbf{S}(n-1)\mathbf{\Lambda}] \quad (\text{B.43})$$

with $\mathbf{S}(0) = \mathbf{I}$. From the recursion in Eq. (B.42), we see that we must require that $\mathbf{c}^T(0)\mathbf{S}(n)\mathbf{c}(0)$ remains bounded for $n \rightarrow \infty$, regardless of the initial conditions $\mathbf{c}(0)$. Since $\mathbf{c}^T(0)\mathbf{S}(n)\mathbf{c}(0)$ remains bounded if and only if all the eigenvalues of $\mathbf{S}(n)$ are in the interval $[-1, 1]$, we therefore perform an eigenvalue analysis of $\mathbf{S}(n)$.

Eigenvalue Analysis of $\mathbf{S}(n)$

Define the vector of ones

$$\mathbf{1} = [1 \quad 1 \quad \cdots \quad 1]^T . \quad (\text{B.44})$$

Using this vector, we may write the trace of a diagonal matrix $\mathbf{M}$ as

$$\text{tr}(\mathbf{M}) = \mathbf{1}^T \mathbf{M} \mathbf{1} . \quad (\text{B.45})$$

Since $\mathbf{S}(n)$ and $\mathbf{\Lambda}$ are diagonal matrices, the product $\mathbf{S}(n)\mathbf{\Lambda}$ is also diagonal, and we have that

$$\mathbf{S}(n-1)\mathbf{\Lambda} = \mathbf{\Lambda}\mathbf{S}(n-1) \quad (\text{B.46})$$

$$\text{tr}(\mathbf{S}(n)\mathbf{\Lambda}) = \mathbf{1}^T \mathbf{S}(n)\mathbf{\Lambda} \mathbf{1} = \mathbf{1}^T \mathbf{\Lambda}\mathbf{S}(n)\mathbf{1} . \quad (\text{B.47})$$

Multiplying both sides of Eq. (B.43) from the right with $\mathbf{1}$, we obtain

$$\mathbf{S}(n)\mathbf{1} = \mathbf{S}(n-1)\mathbf{1} - 2\mu\mathbf{\Lambda}\mathbf{S}(n-1)\mathbf{1} + \mu^2[\mathbf{\Lambda}\mathbf{1}\mathbf{1}^T \mathbf{\Lambda}\mathbf{S}(n-1)\mathbf{1} + 2\mathbf{\Lambda}^2 \mathbf{S}(n-1)\mathbf{1}] \quad (\text{B.48})$$

$$= [\mathbf{I} - 2\mu\mathbf{\Lambda} + \mu^2(\mathbf{\Lambda}\mathbf{1}\mathbf{1}^T \mathbf{\Lambda} + 2\mathbf{\Lambda}^2)]\mathbf{S}(n-1)\mathbf{1} . \quad (\text{B.49})$$

Now, we define

$$\mathbf{D} = \mathbf{I} - 2\mu\mathbf{\Lambda} + \mu^2(\mathbf{\Lambda}\mathbf{1}\mathbf{1}^T\mathbf{\Lambda} + 2\mathbf{\Lambda}^2) \quad (\text{B.50})$$

so that we have

$$\mathbf{S}(n)\mathbf{1} = \mathbf{D}\mathbf{S}(n-1)\mathbf{1} \quad (\text{B.51})$$

$$= \mathbf{D}^2\mathbf{S}(n-2)\mathbf{1} \quad (\text{B.52})$$

$$\vdots$$

$$= \mathbf{D}^n\mathbf{S}(0)\mathbf{1} \quad (\text{B.53})$$

This means that the elements of the diagonal matrix $\mathbf{S}(n)$ are bounded if and only if all the eigenvalues of $\mathbf{D}$ are in the interval $[-1, 1]$.

Eigenvalue Analysis of $\mathbf{D}$

We can write $\mathbf{D}$ as

$$\mathbf{D} = 2(\mathbf{I} - \mu\mathbf{\Lambda})^2 + 2\mu\mathbf{\Lambda} + \mu^2\mathbf{\Lambda}\mathbf{1}\mathbf{1}^T\mathbf{\Lambda} \quad (\text{B.54})$$

which is clearly positive semidefinite since $\mu > 0$ and $\lambda_m > 0$. Thus, all eigenvalues of $\mathbf{D}$ are non-negative, and the only requirement to the eigenvalues is therefore that they are smaller than one. This is equivalent to requiring that $\mathbf{I} - \mathbf{D}$ is positive definite. From Sylvester's criterion, we know that an $M \times M$ matrix $\mathbf{A}$ is positive definite if and only if the determinant of all the upper $m \times m$ matrices $\mathbf{A}_m$ of $\mathbf{A}$ are positive for $m = 1, \dots, M$. If $\mathbf{1}_m$ is a vector consisting of m ones, $\mathbf{I}_m$ is the $m \times m$ identity matrix, and $\mathbf{\Lambda}_m$ and $\mathbf{D}_m$ are the upper $m \times m$ matrices of $\mathbf{\Lambda}$ and $\mathbf{D}$, respectively, we have that

$$\mathbf{D}_m = \mathbf{I}_m - 2\mu\mathbf{\Lambda}_m + \mu^2\mathbf{\Lambda}_m(\mathbf{1}_m\mathbf{1}_m^T + 2\mathbf{I}_m)\mathbf{\Lambda}_m, \quad \text{for } m = 1, \dots, M, \quad (\text{B.55})$$

and we require that

$$|\mathbf{I}_m - \mathbf{D}_m| > 0, \quad \text{for } m = 1, \dots, M \quad (\text{B.56})$$

for $\mathbf{I} - \mathbf{D}$ to be positive definite. The determinant of $\mathbf{I}_m - \mathbf{D}_m$ is

$$|\mathbf{I}_m - \mathbf{D}_m| = |2\mu\mathbf{\Lambda}_m - \mu^2\mathbf{\Lambda}_m(\mathbf{1}_m\mathbf{1}_m^T + 2\mathbf{I}_m)\mathbf{\Lambda}_m| \quad (\text{B.57})$$

$$= |\mu^2\mathbf{\Lambda}_m| |2\mu^{-1}\mathbf{I}_m - 2\mathbf{\Lambda}_m - \mathbf{1}_m\mathbf{1}_m^T\mathbf{\Lambda}_m| \quad (\text{B.58})$$

Since $\mu > 0$ and $\lambda_m > 0$, the determinant of $\mu^2\mathbf{\Lambda}_m$ is positive for all $m = 1, \dots, M$. In order to evaluate the second determinant, we use the matrix determinant lemma which states that

$$|\mathbf{A} + \mathbf{U}\mathbf{V}^T| = |\mathbf{A}| |\mathbf{I} + \mathbf{V}^T\mathbf{A}^{-1}\mathbf{U}| \quad (\text{B.59})$$

where $\mathbf{A}$ is an $N \times N$ matrix, and $\mathbf{U}$ and $\mathbf{V}$ are $N \times M$ matrices. From the matrix determinant lemma, we have that

$$|2\mu^{-1}\mathbf{I}_m - 2\mathbf{\Lambda}_m - \mathbf{1}_m\mathbf{1}_m^T\mathbf{\Lambda}_m| = |2\mu^{-1}\mathbf{I}_m - 2\mathbf{\Lambda}_m| \left| 1 - \frac{1}{2}\mathbf{1}_m^T\mathbf{\Lambda}_m(\mu^{-1}\mathbf{I}_m - \mathbf{\Lambda}_m)^{-1}\mathbf{1}_m \right| \quad (\text{B.60})$$

The argument of the first determinant is a diagonal matrix. Thus, it leads to

$$0 < 2\mu^{-1} - 2\lambda_m, \quad \text{for } m = 1, \dots, M \quad (\text{B.61})$$

$\Updownarrow$

$$\mu < \frac{1}{\lambda_{\max}}. \quad (\text{B.62})$$

The argument of the second determinant is a scalar. It leads to

$$0 < 1 - \frac{1}{2} \mathbf{1}_m^T \mathbf{\Lambda}_m (\mu^{-1} \mathbf{I}_m - \mathbf{\Lambda}_m)^{-1} \mathbf{1}_m, \quad \text{for } m = 1, \dots, M \quad (\text{B.63})$$

$\Updownarrow$

$$1 > \frac{1}{2} \mathbf{1}_m^T \mathbf{\Lambda}_m (\mu^{-1} \mathbf{I}_m - \mathbf{\Lambda}_m)^{-1} \mathbf{1}_m, \quad \text{for } m = 1, \dots, M \quad (\text{B.64})$$

$$= \frac{1}{2} \text{tr}(\mathbf{\Lambda}_m (\mu^{-1} \mathbf{I}_m - \mathbf{\Lambda}_m)^{-1}), \quad \text{for } m = 1, \dots, M \quad (\text{B.65})$$

$$= \frac{1}{2} \sum_{i=1}^m \frac{\lambda_i}{\mu^{-1} - \lambda_i}, \quad \text{for } m = 1, \dots, M \quad (\text{B.66})$$

$$= \frac{\mu}{2} \sum_{i=1}^m \frac{\lambda_i}{1 - \mu \lambda_i} = f_m(\mu), \quad \text{for } m = 1, \dots, M. \quad (\text{B.67})$$

The first bound on the step-size in Eq. (B.62) ensures that $\lambda_i/(1 - \mu \lambda_i)$ is always positive. Thus,

$$f_1(\mu) < f_2(\mu) < \dots < f_M(\mu), \quad \text{for } \mu \in [0, \lambda_{\max}^{-1}]. \quad (\text{B.68})$$

Therefore, if the step-size satisfies

$$f(\mu) = f_M(\mu) < 1, \quad (\text{B.69})$$

then all of the functions $f_m(\mu)$ are also smaller than one. Moreover, $f(0) = 0$, and $f(\mu)$ is an increasing function as long as the first bound on the step-size in Eq. (B.62) is satisfied. The latter follows since the derivative of $f(\mu)$ satisfies

$$\frac{df}{d\mu} = \frac{1}{2} \sum_{m=1}^M \frac{\lambda_m}{(1 - \mu \lambda_m)^2} > 0, \quad \text{for } \mu \in [0, \lambda_{\max}^{-1}]. \quad (\text{B.70})$$

These observations lead to the conclusion that $\mathbf{I} - \mathbf{D}$ is positive definite, provided that the step-size satisfies the bound

$$\boxed{f(\mu) = \frac{\mu}{2} \sum_{m=1}^M \frac{\lambda_m}{1 - \mu \lambda_m} < 1}. \quad (\text{B.71})$$

In matrix notation, we can write this bound as

$$\boxed{f(\mu) = \frac{1}{2} \mathbf{1}^T \mathbf{\Lambda} (\mu^{-1} \mathbf{I} - \mathbf{\Lambda})^{-1} \mathbf{1} = \frac{1}{2} \text{tr}(\mathbf{\Lambda} (\mu^{-1} \mathbf{I} - \mathbf{\Lambda})^{-1}) < 1} \quad (\text{B.72})$$

where the last equality follows since $\mathbf{\Lambda} (\mu^{-1} \mathbf{I} - \mathbf{\Lambda})^{-1}$ is diagonal. Moreover, since

$$\lim_{\mu \rightarrow \lambda_{\max}^{-1}} f(\mu) = \infty > 1, \quad (\text{B.73})$$

the first bound in Eq. (B.62) is always satisfied if the second bound in Eq. (B.71) is satisfied. Thus, we have shown that the LMS algorithm converges in the mean-square if and only if the step-size satisfies Eq. (B.71) or, equivalently, Eq. (B.72).

B.3.2 Learning Curve

The value of the cost function at time n is given by

$$J_1(\mathbf{w}(n)) = E[e^2(n)] = E[(v(n) + \mathbf{u}^T(n)\Delta\mathbf{w}(n))^2] \quad (\text{B.74})$$

$$= E[\Delta\mathbf{w}^T(n)\mathbf{u}(n)\mathbf{u}^T(n)\Delta\mathbf{w}(n)] + J_{\min} \quad (\text{B.75})$$

where the last equality follows from the fact that $v(n)$ and $\mathbf{u}(n)$ are uncorrelated. If we also use the law of iterated expectations and that $\mathbf{u}(n)$ and $\mathbf{w}(n)$ are uncorrelated, we obtain that

$$J_1(\mathbf{w}(n)) = E[\Delta\mathbf{w}^T(n)E[\mathbf{u}(n)\mathbf{u}^T(n)|\mathbf{w}(n)]\Delta\mathbf{w}(n)] + J_{\min} \quad (\text{B.76})$$

$$= E[\Delta\mathbf{w}^T(n)E[\mathbf{u}(n)\mathbf{u}^T(n)]\Delta\mathbf{w}(n)] + J_{\min} \quad (\text{B.77})$$

$$= E[\Delta\mathbf{w}^T(n)\mathbf{R}_u\Delta\mathbf{w}(n)] + J_{\min} . \quad (\text{B.78})$$

Finally, if we replace the correlation matrix $\mathbf{R}_u$ of $\mathbf{u}(n)$ with its eigenvalue decomposition $\mathbf{R}_u = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^T$, we have that

$$J_1(\mathbf{w}(n)) = E[\mathbf{c}^T(n)\mathbf{\Lambda}\mathbf{c}(n)] + J_{\min} \quad (\text{B.79})$$

where we again have defined that $\mathbf{c}(n) = \mathbf{X}^T\Delta\mathbf{w}(n)$. Now, for $\mathbf{S}(0) = \mathbf{\Lambda}$, we obtain from Eq. (B.42) that

$$J_1(\mathbf{w}(n)) = \mathbf{c}^T(0)\mathbf{S}(n)\mathbf{c}(0) + \mu^2 J_{\min} \sum_{i=0}^{n-1} \text{tr}(\mathbf{S}(i)\mathbf{\Lambda}) + J_{\min} \quad (\text{B.80})$$

where

$$\mathbf{S}(n) = \mathbf{S}(n-1) - 2\mu\mathbf{\Lambda}\mathbf{S}(n-1) + \mu^2[\mathbf{\Lambda}\text{tr}(\mathbf{S}(n-1)\mathbf{\Lambda}) + 2\mathbf{\Lambda}\mathbf{S}(n-1)\mathbf{\Lambda}] \quad (\text{B.81})$$

$$= \text{diag}(\mathbf{S}(n)\mathbf{1}) = \text{diag}(\mathbf{D}^n\mathbf{\Lambda}\mathbf{1}) . \quad (\text{B.82})$$

Here, $\text{diag}(\cdot)$ creates a diagonal matrix from a vector.

B.4 Steady-State Analysis

In the steady-state analysis, we derive expressions for the mean-square deviation (MSD), the excess mean-square error (EMSE), and the misadjustment of the LMS algorithm. These expressions can be derived in two different ways. One way is presented in [7, pp. 462–465]. Here, we give another derivation which we believe is more intuitive.

B.4.1 Mean-Square Deviation

The MSD is given by the limit

$$\text{MSD:} \quad \lim_{n \rightarrow \infty} E[\|\Delta\mathbf{w}(n)\|^2] , \quad (\text{B.83})$$

and we denote this limit by $E[\|\Delta \mathbf{w}(\infty)\|^2]$. We make direct use of the definition in the derivation of the MSD. Again, we define $\mathbf{c}(n) = \mathbf{X}^T \Delta \mathbf{w}(n)$. Thus, from Eq. (B.42), we have that

$$E[\|\Delta \mathbf{w}(\infty)\|^2] = \lim_{n \rightarrow \infty} E[\|\Delta \mathbf{w}(n)\|^2] = \lim_{n \rightarrow \infty} E[\|\mathbf{c}(n)\|^2] \quad (\text{B.84})$$

$$= \lim_{n \rightarrow \infty} \left\{ \mathbf{c}^T(0) \mathbf{S}(n) \mathbf{c}(0) + \mu^2 J_{\min} \sum_{i=0}^{n-1} \text{tr}(\mathbf{S}(i) \mathbf{\Lambda}) \right\} \quad (\text{B.85})$$

$$= \lim_{n \rightarrow \infty} \mathbf{c}^T(0) \mathbf{S}(n) \mathbf{c}(0) + \lim_{n \rightarrow \infty} \mu^2 J_{\min} \sum_{i=0}^{n-1} \text{tr}(\mathbf{S}(i) \mathbf{\Lambda}) . \quad (\text{B.86})$$

with $\mathbf{S}(0) = \mathbf{I}$. If we select the step-size μ such that the LMS algorithm converges in the mean-square, the first term equals zero. Thus, we have that

$$E[\|\Delta \mathbf{w}(\infty)\|^2] = \lim_{n \rightarrow \infty} \mu^2 J_{\min} \sum_{i=0}^{n-1} \text{tr}(\mathbf{S}(i) \mathbf{\Lambda}) = \mu^2 J_{\min} \sum_{i=0}^{\infty} \text{tr}(\mathbf{S}(i) \mathbf{\Lambda}) . \quad (\text{B.87})$$

The matrices $\mathbf{S}(i)$ and $\mathbf{\Lambda}$ are both diagonal, and we therefore use Eq. (B.47) to obtain

$$E[\|\Delta \mathbf{w}(\infty)\|^2] = \mu^2 J_{\min} \sum_{i=0}^{\infty} \mathbf{1}^T \mathbf{\Lambda} \mathbf{S}(i) \mathbf{1} \quad (\text{B.88})$$

$$= \mu^2 J_{\min} \sum_{i=0}^{\infty} \mathbf{1}^T \mathbf{\Lambda} \mathbf{D}^i \mathbf{1} \quad (\text{B.89})$$

$$= \mu^2 J_{\min} \mathbf{1}^T \mathbf{\Lambda} \sum_{i=0}^{\infty} [\mathbf{D}^i] \mathbf{1} \quad (\text{B.90})$$

where the second equality follows from Eq. (B.53) with $\mathbf{S}(0) = \mathbf{I}$ and $\mathbf{D}$ defined in Eq. (B.50). Since all the eigenvalues of $\mathbf{D}$ have a magnitude smaller than 1, we have from the geometric series of matrices that [12, p. 58]

$$\sum_{i=0}^{\infty} \mathbf{D}^i = (\mathbf{I} - \mathbf{D})^{-1} . \quad (\text{B.91})$$

Thus, we obtain that

$$E[\|\Delta \mathbf{w}(\infty)\|^2] = \mu^2 J_{\min} \mathbf{1}^T \mathbf{\Lambda} (\mathbf{I} - \mathbf{D})^{-1} \mathbf{1} \quad (\text{B.92})$$

$$= \mu^2 J_{\min} \mathbf{1}^T \mathbf{\Lambda} (2\mu \mathbf{\Lambda} - \mu^2 (\mathbf{\Lambda} \mathbf{1} \mathbf{1}^T \mathbf{\Lambda} + 2\mathbf{\Lambda}^2))^{-1} \mathbf{1} \quad (\text{B.93})$$

$$= \mu^2 J_{\min} \mathbf{1}^T \mathbf{\Lambda} [(2\mu^{-1} \mathbf{I} - 2\mathbf{\Lambda} - \mathbf{\Lambda} \mathbf{1} \mathbf{1}^T) \mu^2 \mathbf{\Lambda}]^{-1} \mathbf{1} \quad (\text{B.94})$$

$$= J_{\min} \mathbf{1}^T (2\mu^{-1} \mathbf{I} - 2\mathbf{\Lambda} - \mathbf{\Lambda} \mathbf{1} \mathbf{1}^T)^{-1} \mathbf{1} . \quad (\text{B.95})$$

Now, by defining

$$\mathbf{G} = \mu^{-1} \mathbf{I} - \mathbf{\Lambda} \quad (\text{B.96})$$

and using the matrix inversion lemma from Eq. (4.5), we obtain

$$E[\|\Delta \mathbf{w}(\infty)\|^2] = J_{\min} \mathbf{1}^T (2\mathbf{G} - \mathbf{\Lambda} \mathbf{1} \mathbf{1}^T)^{-1} \mathbf{1} \quad (\text{B.97})$$

$$= J_{\min} \mathbf{1}^T \left[\frac{1}{2} \mathbf{G}^{-1} + \frac{1}{2} \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \left(1 - \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \right)^{-1} \mathbf{1}^T \mathbf{G}^{-1} \frac{1}{2} \right] \mathbf{1} \quad (\text{B.98})$$

$$= J_{\min} \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{1} \left[1 + \left(1 - \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \right)^{-1} \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \right] \quad (\text{B.99})$$

$$= J_{\min} \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{1} \left(1 - \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \right)^{-1}. \quad (\text{B.100})$$

Finally, from Eq. (B.72), we have that

$$f(\mu) = \frac{1}{2} \mathbf{1}^T \mathbf{\Lambda} \mathbf{G}^{-1} \mathbf{1} = \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \quad (\text{B.101})$$

which leads to

$$E[\|\Delta \mathbf{w}(\infty)\|^2] = \frac{J_{\min}}{1 - f(\mu)} \frac{\mu}{2} \sum_{m=1}^M \frac{1}{1 - \mu \lambda_m}. \quad (\text{B.102})$$

B.4.2 Excess Mean-Square Error

The derivation of the EMSE is done in the same way as the MSE was derived. The EMSE is given by the limit

$$\text{EMSE:} \quad \lim_{n \rightarrow \infty} J_1(\mathbf{w}(n)) - J_{\min}, \quad (\text{B.103})$$

and we denote it by J_{ex} . Inserting the expression for the learning curve from Eq. (B.80) in this limit yields

$$J_{\text{ex}} = \lim_{n \rightarrow \infty} \left\{ \mathbf{c}^T(0) \mathbf{S}(n) \mathbf{c}(0) + \mu^2 J_{\min} \sum_{i=0}^{n-1} \text{tr}(\mathbf{S}(i) \mathbf{\Lambda}) \right\} \quad (\text{B.104})$$

$$= \lim_{n \rightarrow \infty} \mathbf{c}^T(0) \mathbf{S}(n) \mathbf{c}(0) + \lim_{n \rightarrow \infty} \mu^2 J_{\min} \sum_{i=0}^{n-1} \text{tr}(\mathbf{S}(i) \mathbf{\Lambda}) \quad (\text{B.105})$$

where $\mathbf{S}(0) = \mathbf{\Lambda}$ and $\mathbf{c}(n) = \mathbf{X}^T \Delta \mathbf{w}(n)$. If we select the step-size μ such that the LMS algorithm converges in the mean-square, the first term equals zero. Thus, we have that

$$J_{\text{ex}} = \lim_{n \rightarrow \infty} \mu^2 J_{\min} \sum_{i=0}^{n-1} \text{tr}(\mathbf{S}(i) \mathbf{\Lambda}) = \mu^2 J_{\min} \sum_{i=0}^{\infty} \text{tr}(\mathbf{S}(i) \mathbf{\Lambda}). \quad (\text{B.106})$$

Note that the expression for the EMSE is the same as for the MSD, except for the value of $\mathbf{S}(0)$. The matrices $\mathbf{S}(i)$ and $\mathbf{\Lambda}$ are both diagonal, and we therefore use Eq. (B.47) to obtain

$$J_{\text{ex}} = \mu^2 J_{\min} \sum_{i=0}^{\infty} \mathbf{1}^T \mathbf{\Lambda} \mathbf{S}(i) \mathbf{1} \quad (\text{B.107})$$

$$= \mu^2 J_{\min} \sum_{i=0}^{\infty} \mathbf{1}^T \mathbf{\Lambda} \mathbf{D}^i \mathbf{\Lambda} \mathbf{1} \quad (\text{B.108})$$

$$= \mu^2 J_{\min} \mathbf{1}^T \mathbf{\Lambda} \sum_{i=0}^{\infty} [\mathbf{D}^i] \mathbf{\Lambda} \mathbf{1} \quad (\text{B.109})$$

where the second equality follows from Eq. (B.53) with $\mathbf{S}(0) = \mathbf{\Lambda}$ and $\mathbf{D}$ defined in Eq. (B.50). Since all the eigenvalues of $\mathbf{D}$ have a magnitude smaller than 1, we have from the geometric series of matrices that [12, p. 58]

$$\sum_{i=0}^{\infty} \mathbf{D}^i = (\mathbf{I} - \mathbf{D})^{-1}. \quad (\text{B.110})$$

Thus, we obtain that

$$J_{\text{ex}} = \mu^2 J_{\min} \mathbf{1}^T \mathbf{\Lambda} (\mathbf{I} - \mathbf{D})^{-1} \mathbf{\Lambda} \mathbf{1} \quad (\text{B.111})$$

$$= \mu^2 J_{\min} \mathbf{1}^T \mathbf{\Lambda} (2\mu \mathbf{\Lambda} - \mu^2 (\mathbf{\Lambda} \mathbf{1} \mathbf{1}^T \mathbf{\Lambda} + 2\mathbf{\Lambda}^2))^{-1} \mathbf{\Lambda} \mathbf{1} \quad (\text{B.112})$$

$$= \mu^2 J_{\min} \mathbf{1}^T \mathbf{\Lambda} [(2\mu^{-1} \mathbf{I} - 2\mathbf{\Lambda} - \mathbf{\Lambda} \mathbf{1} \mathbf{1}^T) \mu^2 \mathbf{\Lambda}]^{-1} \mathbf{\Lambda} \mathbf{1} \quad (\text{B.113})$$

$$= J_{\min} \mathbf{1}^T (2\mu^{-1} \mathbf{I} - 2\mathbf{\Lambda} - \mathbf{\Lambda} \mathbf{1} \mathbf{1}^T)^{-1} \mathbf{\Lambda} \mathbf{1}. \quad (\text{B.114})$$

Now, by defining

$$\mathbf{G} = \mu^{-1} \mathbf{I} - \mathbf{\Lambda} \quad (\text{B.115})$$

and using the matrix inversion lemma from Eq. (4.5), we obtain

$$J_{\text{ex}} = J_{\min} \mathbf{1}^T (2\mathbf{G} - \mathbf{\Lambda} \mathbf{1} \mathbf{1}^T)^{-1} \mathbf{\Lambda} \mathbf{1} \quad (\text{B.116})$$

$$= J_{\min} \mathbf{1}^T \left[\frac{1}{2} \mathbf{G}^{-1} + \frac{1}{2} \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \left(1 - \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \right)^{-1} \mathbf{1}^T \mathbf{G}^{-1} \frac{1}{2} \right] \mathbf{\Lambda} \mathbf{1} \quad (\text{B.117})$$

$$= J_{\min} \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \left[1 + \left(1 - \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \right)^{-1} \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \right] \quad (\text{B.118})$$

$$= J_{\min} \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \left(1 - \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \right)^{-1}. \quad (\text{B.119})$$

Finally, from Eq. (B.72), we have that

$$f(\mu) = \frac{1}{2} \mathbf{1}^T \mathbf{\Lambda} \mathbf{G}^{-1} \mathbf{1} = \frac{1}{2} \mathbf{1}^T \mathbf{G}^{-1} \mathbf{\Lambda} \mathbf{1} \quad (\text{B.120})$$

which leads to

$$\boxed{J_{\text{ex}} = J_{\min} \frac{f(\mu)}{1 - f(\mu)}}. \quad (\text{B.121})$$

B.4.3 Misadjustment

The expression for the misadjustment is

$$\boxed{\mathcal{M} = \frac{J_{\text{ex}}}{J_{\min}} = \frac{f(\mu)}{1 - f(\mu)}}. \quad (\text{B.122})$$